Monitoring of wastewater treatment plants using neural networks

by

Jacob Johannes Mostert

Thesis presented in partial fulfilment of the requirements for the degree of Doctor of Philosophy (Electronic Engineering) in the Faculty of Engineering at Stellenbosch University

Supervisor: Prof. Thomas Niesler

2026

Someone.

Acknowledgements

* Supervisor X.
* Funding Y.

> *A very long quote*
> *from someone very important.*
> – Mr M

Thank you to Leanne for helping to prepare this template.

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: 10 May 2024

Copyright © 2026 Stellenbosch University
All rights reserved.

Abstract

Uittreksel

Table of Contents

Abstract **iii**

Uittreksel **iv**

Table of contents **v**

List of figures **ix**

List of tables **xi**

Nomenclature **xvi**

Acronyms and abbreviations xvii

1. Introduction **1**

1.1. Problem Statement 1

1.2. Objectives 1

1.3. Scope 2

1.4. Report Structure 2

2. Background **3**

2.1. Wastewater Treatment Works 3

2.2. Convolutional Neural Networks 3

2.2.1. Building Blocks of a Convolutional Neural Network 3

2.2.2. Residual Networks: ResNet-18 and ResNet-50 6

2.2.3. Densely Connected Networks: DenseNet-121 7

2.2.4. Inception Networks: Inception-v3 9

2.2.5. Efficient Networks: EfficientNet-B0 9

2.2.6. ConvNeXt-Tiny 11

2.2.7. Summary of Architectures 12

2.3. Evaluation Metrics for Classification 13

2.4. Gradient-Weighted Class Activation Mapping 14

3. Literature Review **15**

3.1. The Monitoring Problem in South Africa 15

3.2. Reading Process State from Images Inside the Plant 15

3.3. Satellite Remote Sensing of Water Quality 16

3.4. Remote Sensing of Treatment Plants 17

3.5. Pretrained CNNs on a Small, Texture-Defined Problem 17

3.6. Testing on Unseen Plants and Checking What the Model Sees 18
3.7. Summary and Conclusions 19

4. Dataset **20**
4.1. WWTW Sites 20
4.2. Annotations 21
4.3. Facility Structure 23
4.4. Data Preparation 24
4.5. Excluded from the dataset 26
4.6. Data Splitting 26
4.6.1. Leave-one-out Split 26
4.6.2. Stratified Train/Development Split 27
4.7. Relabelling 28
4.8. Final Dataset 30

5. Experimental Design **31**
5.1. Overview 31
5.2. Model Architectures and Transfer Learning 32
5.3. Augmentation 34
5.4. Class Imbalance 35
5.5. Hyperparameter Tuning 38
5.5.1. Why Optuna 38
5.5.2. Search Procedure 38
5.5.3. AlexNet 38
5.6. Training Procedure 39
5.7. Experiment 1: Mixed-Facility Baseline 41
5.8. Experiment 2: Leave-One-Facility-Out Evaluation 42
5.9. Experiment 3: Binary (OK / Not-OK) Reformulation 44
5.10. Evaluation Metrics 45
5.11. Qualitative Analysis: Grad-CAM 46
5.11.1. Method 46
5.11.2. Models and Example Selection 46

6. Results **47**
6.1. Overview 47
6.2. Experiment 1: Mixed-Facility Baseline 47
6.2.1. Clarifiers 47
6.2.1.1. Comparison Table 47
6.2.1.2. Accuracy and AUC 47
6.2.1.3. Per-Class Precision, Recall and F1 47

6.2.2. Aerobic Zones . . . 48
6.2.2.1. Comparison Table . . . 48
6.2.2.2. Accuracy and AUC . . . 48
6.2.2.3. Per-Class Precision, Recall and F1 . . . 48
6.2.3. Effect of Augmentation . . . 48
6.3. Experiment 2: Leave-One-Facility-Out Evaluation . . . 50
6.3.1. Clarifiers . . . 50
6.3.1.1. Comparison Table . . . 50
6.3.1.2. Accuracy and AUC Across Facilities . . . 50
6.3.1.3. Per-Class Precision, Recall and F1 . . . 50
6.3.1.4. Training Curves and Confusion Matrices . . . 50
6.3.2. Aerobic Zones . . . 54
6.3.2.1. Comparison Table . . . 54
6.3.2.2. Accuracy and AUC Across Facilities . . . 54
6.3.2.3. Per-Class Precision, Recall and F1 . . . 54
6.3.2.4. Training Curves and Confusion Matrices . . . 54
6.3.3. Generalisation: Mixed-Facility Baseline vs Leave-One-Facility-Out 57
6.3.4. Hue Shift . . . 57
6.4. Experiment 3: Binary (OK / Not-OK) Reformulation . . . 58
6.4.1. Clarifiers . . . 58
6.4.2. Aerobic Zones . . . 58
6.4.3. Choosing an Operating Threshold . . . 61
6.4.4. Threshold Robustness Across Facilities . . . 64
6.5. Qualitative Analysis: Grad-CAM . . . 65
6.5.1. Clarifiers . . . 65
6.5.2. Aerobic Zones . . . 67
6.5.3. Summary of Observations . . . 68
6.6. Summary of Findings . . . 69

7. Summary and conclusions **70**

References **71**

A. Hyperparameter Search Trials **76**
A.1. Clarifiers . . . 76
A.1.1. ResNet-18 . . . 76
A.1.2. EfficientNet-B0 . . . 76
A.1.3. DenseNet-121 . . . 77
A.1.4. ResNet-50 . . . 78
A.1.5. ConvNeXt-Tiny . . . 78

A.1.6. Inception-v3 79
A.2. Aerobic Zones 80
A.2.1. ResNet-18 80
A.2.2. EfficientNet-B0 80
A.2.3. DenseNet-121 81
A.2.4. ResNet-50 82
A.2.5. ConvNeXt-Tiny 82
A.2.6. Inception-v3 83

List of Figures

2.1. Visual comparison of functional and dysfunctional wastewater treatment components. 4

4.1. Number of labelled images per month for each of the five facilities. Cape Flats and Waterval have a separate image for each unit on most dates, so their cells can be larger than one. 20
4.2. Location of the five WWTW facilities in the dataset. 21
4.3. Overview of the Atlantis, Cape Flats, Waterval, Fisantekraal, and Northern Works WWTW facilities. 22
4.4. One example of each aerobic zone class, with the facility it comes from. Each example is the test image of the stratified split (Chapter 5) that all six pretrained architectures classified correctly with the highest mean confidence, so that it is a clear case of its class. 23
4.5. One example of each clarifier class, with the facility it comes from, chosen in the same way as in Figure 4.4. The area outside each tank is masked to black by the preparation script (Section 4.4). 23
4.6. Number of clarifier images from each facility in the training, development and test sets when Waterval is held out. The test set contains only Waterval, while the remaining four facilities are split into training and development sets. 27
4.7. A relabelled clarifier from Cape Flats, shown with some of its surroundings. The original label was “Scum”, but the label was corrected to “Empty”, since the tank appears to be drained, with its floor and the shadow of the bridge visible inside it. 28
4.8. The relabelling interface, showing the original label and the per-class correction buttons. 29

5.1. End-to-end experimental pipeline, from labelled satellite imagery to a saved, comparable results file per combination of component, architecture and test set. 32
5.2. Augmentations applied to a single **Scum** clarifier from Cape Flats. Top row: the original image, followed by a horizontal flip, a rotation of 15°, and a random colour jitter, each applied on its own. Bottom row: four random outputs of the full augmentation pipeline (flip, rotation and colour jitter combined). 36

5.3. The training loop used for every architecture: an epoch of training and development-set evaluation, a scheduler step, a checkpoint decision based on development loss, and a stopping check repeated every epoch until training ends. 40

6.1. Test accuracy and mean one-vs-rest AUC of every clarifier architecture on the stratified split (Experiment 1). 48
6.2. Test accuracy and mean one-vs-rest AUC of every aerobic zone architecture on the stratified split (Experiment 1). 50
6.3. Test accuracy and mean one-vs-rest AUC of every clarifier architecture for each held-out facility, and the mean across facilities. 51
6.4. Training and development loss and accuracy of the ConvNeXt-Tiny clarifier model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint. 52
6.5. Confusion matrices of the ConvNeXt-Tiny clarifier model on each held-out facility. 53
6.6. Test accuracy and mean one-vs-rest AUC of every aerobic zone architecture for each held-out facility, and the mean across facilities. 54
6.7. Training and development loss and accuracy of the DenseNet-121 aerobic zone model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint. 56
6.8. Confusion matrices of the DenseNet-121 aerobic zone model on each held-out facility. 56
6.9. Binary OK / Not-OK ROC curves of the clarifier models. The dotted line marks the 95% target recall. The legend in (b) gives the population standard deviation, so it differs slightly from Table 6.14. 59
6.10. Binary OK / Not-OK ROC curves of the aerobic zone models. The dotted line marks the 95% target recall. 60
6.11. Row-normalised binary confusion matrices of the clarifier models at the chosen threshold. 62
6.12. Row-normalised binary confusion matrices of the aerobic zone models at the chosen threshold. 63
6.13. Grad-CAM heatmaps of the ConvNeXt-Tiny clarifier model, for test images from Fisantekraal, which was held out during training. Top: one correctly classified clarifier per class. Bottom: one clarifier from each of the four most common mistakes. Every heatmap is calculated for the predicted class, and p is the softmax probability of that class. Red regions contributed the most to the prediction. 66

6.14. Grad-CAM heatmaps of the DenseNet-121 aerobic zone model, for test images from Waterval, which was held out during training. Top: one correctly classified aerobic zone per class. Bottom: one aerobic zone from each of the three most common mistakes. Every heatmap is calculated for the predicted class, and p is the softmax probability of that class. Red regions contributed the most to the prediction. 68

List of Tables

2.1. Stage-by-stage structure of ResNet-18 and ResNet-50 for a 224×224 input. Bracketed blocks are repeated the stated number of times, with the layers inside each bracket applied in sequence. FC: fully-connected layer. 8

2.2. Growth in the number of feature-map channels available to each layer within DenseNet-121's first dense block ($k_0 = 64$ input channels, growth rate $k = 32$). 8

2.3. Branches of a representative Inception-v3 module. Each branch is applied to the same input feature map, and their outputs are concatenated channel-wise to form the module's output. 9

2.4. Stage-by-stage structure of EfficientNet-B0 for a 224×224 input. 10

2.5. Operation-by-operation comparison of a ResNet bottleneck block and a ConvNeXt block. Both blocks add their input back onto their output via a residual connection. 12

2.6. Summary of the seven architectures used in this project, ordered by year of publication, with their parameter counts in millions. The clarifier and aerobic zone columns include the tuned classifier head of each final model. Inception-v3's totals include its auxiliary classifier, which is only used during training. 13

4.1. Number of units and labelled images per facility. 24

4.2. Total number of clarifier regions per class, per facility, before and after relabelling. "Old" is the count in the original annotations; "New" is the count after relabelling and the supervisor's review. "Relabels" is the number of regions at that facility whose final label differs from the original one. . . . 29

4.3. Summary of image counts per class for the Aerobic Zones. 30

4.4. Summary of image counts per class for the Clarifiers. 30

5.1. Parameter count of each of the seven architectures evaluated, from smallest to largest, as defined in torchvision with the original 1000-class ImageNet classifier. The new classifier heads (Section 5.5) change these totals. 33

5.2. Number of training images per clarifier class and the resulting class weight w_c, for each held-out test facility and for the stratified split. 37

5.3. Number of training images per aerobic zone class and the resulting class weight w_c, for each held-out test facility and for the stratified split. 37

5.4. Search range of each tuned hyperparameter, and the values used for each architecture and component, with the development accuracy of the selected trial. The learning rate is sampled log-uniformly, and the batch size from $\{8, 16, 24, 32, 48, 64, 96, 128\}$. H=0 means the classifier head is a single linear layer, and N is unused. The AlexNet values were chosen by hand (Section 5.5.3). 39
5.5. Number of images per class in the training, development and test sets of the stratified 70/20/10 split used in Experiment 1. 41
5.6. Number of training, development and test images in each leave-one-facility-out fold, and the number of test images per class. The test set is the held-out facility, and the training and development sets are drawn from the remaining facilities. 43

6.1. Clarifier test results of every architecture on the stratified 70/20/10 split (Experiment 1). Macro-averaged scores give every class the same weight. The best value in every column is shown in bold. 47
6.2. Per-class precision (P), recall (R) and F1 score of every clarifier architecture on the stratified split (Experiment 1). 47
6.3. Aerobic zone test results of every architecture on the stratified 70/20/10 split (Experiment 1). 48
6.4. Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture on the stratified split (Experiment 1). 49
6.5. Test results on the stratified split with (Aug.) and without (None) augmentation during training. Δ is the difference in percentage points, where a positive value means that augmentation helped. 49
6.6. Clarifier test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the five held-out facilities. 50
6.7. Clarifier test accuracy and mean AUC of every architecture for each held-out facility. 51
6.8. Per-class precision (P), recall (R) and F1 score of every clarifier architecture in Experiment 2, averaged over the held-out facilities at which the class occurs. 51
6.9. Aerobic zone test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the three held-out facilities. 54
6.10. Aerobic zone test accuracy and mean AUC of every architecture for each held-out facility. 55
6.11. Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture in Experiment 2, averaged over the three held-out facilities. 55

6.12. Test accuracy and macro F1 score on the stratified split (Experiment 1) and the mean across held-out facilities (Experiment 2). Δ is the change in percentage points from Experiment 1 to Experiment 2. 57
6.13. Mean clarifier test accuracy across the five held-out facilities, and the total number of correctly classified **Dysfunctional** clarifiers (out of 273), with and without the hue shift. Δ is the change in percentage points when the hue shift is removed. These runs were made before the clarifier labels were reviewed. 58
6.14. Binary OK / Not-OK AUC of every clarifier architecture on the stratified split, and the mean ± standard deviation across held-out facilities. 58
6.15. Binary OK / Not-OK AUC of every aerobic zone architecture on the stratified split, and the mean ± standard deviation across held-out facilities. 59
6.16. Threshold t on the Not-OK score chosen for a 95% recall, with the recall, specificity and precision of the Not-OK class at that threshold. For the leave-one-facility-out split, one threshold is shared by all folds and the metrics are averaged across held-out facilities. 61
6.17. Recall and specificity of the Not-OK class for each held-out facility at the shared clarifier threshold of Table 6.16, with the lowest recall and the standard deviation of the recall across facilities. 64
6.18. Recall and specificity of the Not-OK class for each held-out facility at the shared aerobic zone threshold of Table 6.16, with the lowest recall and the standard deviation of the recall across facilities. 65

A.1. All 20 clarifier Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 76
A.2. All 20 clarifier Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 77
A.3. All 20 clarifier Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 77
A.4. All 20 clarifier Optuna trials for ResNet-50, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 78
A.5. All 20 clarifier Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 79
A.6. All 20 clarifier Optuna trials for Inception-v3, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 79
A.7. All 20 aerobic zone Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 80

A.8. All 20 aerobic zone Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 81
A.9. All 20 aerobic zone Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 81
A.10. All 20 aerobic zone Optuna trials for ResNet-50, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 82
A.11. All 20 aerobic zone Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 83
A.12. All 20 aerobic zone Optuna trials for Inception-v3, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 83

Nomenclature

Acronyms and Abbreviations

MFT My Funky Thesis

Chapter 1

Introduction

1.1 Problem Statement

Currently, the monitoring of wastewater treatment works (WWTW) is accomplished using on-site sensors, physical inspections, or the manual analysis of aerial photography by experienced personnel. While facilities equipped with sensors connected to automated control systems exist globally, they are currently not widely implemented in South Africa. Consequently, South African facilities rely primarily on on-site personnel to monitor operations. These personnel are required to sample the water at various stages of the treatment process at least once a day in an on-site laboratory, and at least once a month, samples should be sent to an independent laboratory for a complete analysis.

However, one or both of these testing protocols do not always occur. Incorrect measurements may be recorded, or tests may be omitted entirely. To detect such problems, physical site visits by officials are currently necessary. These physical inspections are expensive, time-consuming, and impractical to conduct with high frequency. Furthermore, on-site sensors, where utilised, are prone to failure and expensive to install and maintain.

The question this project aims to answer is whether a computer vision system can accurately identify from aerial or satellite images the same visual indicators utilised by trained human operators (such as the characteristic surface turbulence of an aerator or the effluent clarity of a clarifier) to assess performance. Such a system could be utilised to detect problem cases automatically and quickly. Specifically, a discrepancy between the reported laboratory data and the actual visual state observed in the imagery can be used to trigger corrective action. Ultimately, this study investigates the feasibility of leveraging historical satellite imagery to automatically monitor the functional state of Fine Bubble Diffused Aeration (FBDA) systems and clarifiers.

1.2 Objectives

To answer the above question, the project addresses the following three objectives:

1. Prepare image datasets for experimentation and extract features by working with a set of labelled Google Earth satellite images taken on different dates to capture the visual states of WWTW components.

2. Train classifiers to automatically determine the status of the aerobic zones (Functional, Suboptimal, Dysfunctional) and the Clarifiers (Functional, Dysfunctional, Scum, Empty), based on the image data.

3. Evaluate the model's performance by testing it on a separate set of images and comparing its predictions to the known labels, using metrics like precision, recall, F1-score, and area under the ROC curve to quantify how well the model performs in classifying the different states of the WWTW components.

1.3 Scope

This project considers the Fine Bubble Diffused Aeration (FBDA) in aerobic zones, and the secondary clarifiers. The project looks at five WWTW facilities: Atlantis, Cape Flats, Waterval, Fisantekraal and Northern Works. Clarifiers are labelled at all five facilities, and aerobic zones only at Atlantis, Cape Flats and Waterval.

Surface-mounted aerators and anaerobic zones are not considered in this project. All images are obtained from Google Earth, and there is no live satellite feed, no physical water sampling, and no hardware devices deployed on site.

1.4 Report Structure

The remainder of the report is structured as follows:

- Chapter 2 (Background): Describes the WWTW components that are classified in this project, and the CNN architectures that are compared.
- Chapter 3 (Literature Review): Describes existing monitoring solutions for wastewater management, remote sensing technology, and the image processing concepts used in this project.
- Chapter 4 (Dataset): Describes the collection and preprocessing of the image datasets.
- Chapter 5 (Experimental Design): Presents a detailed design of the proposed solution, including the convolutional neural network (CNN) architectures compared for classification, the training procedures and the experiments.
- Chapter 6 (Results): Presents the classification performance metrics and discusses the findings, including any limitations or challenges encountered during the experiments.
- Chapter 7 (Conclusion): Draws conclusions from the results, discusses the limitations of the project, and suggests future work that could improve the system.

Background

2.1 Wastewater Treatment Works

Municipal wastewater treatment plants purify our water using a series of steps. Large facilities typically rely on an effective biological method known as the activated sludge process. This system utilises enormous tanks filled with beneficial microorganisms that naturally break down pollutants. During this process, the microorganisms convert the organic pollutants primarily into carbon dioxide, water, and new cellular biomass. These bacteria require a steady supply of oxygen to survive and work efficiently. To provide this air, facilities often use a technique called Fine Bubble Diffused Aeration (FBDA), which continuously pumps millions of tiny bubbles directly into the water. This happens in a part of the treatment plant known as **aerobic zones**.

Once this active treatment is complete, the mixture flows into large circular settling tanks called **clarifiers**. The dense microbial biomass gradually settles to the bottom of the tank. This settling process leaves clear, treated water at the surface. Rather than discarding this settled biomass, the system continuously circulates it back to the initial tanks. Returning this activated sludge maintains a sufficient concentration of active microorganisms, ensuring the system can efficiently metabolise incoming organic matter.

Interestingly, it is possible to tell a lot about how well these systems are performing just from aerial or satellite photos. A healthy FBDA unit churns the surface into a "leopard print" pattern, while a struggling one looks noticeably smoother. Clarifiers tell a similar story: when they are operating correctly, they separate out the solids and leave behind a clear liquid that looks black from above. When they are operating poorly, solids escape into the effluent and the surface turns green, cloudy, or uneven. Shown in Figure 2.1 are examples of functional and dysfunctional aerators and clarifiers, highlighting the visual cues that indicate their performance levels.

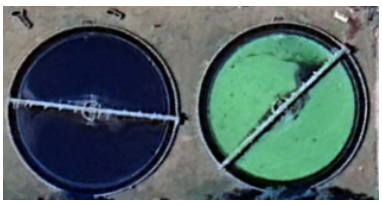
2.2 Convolutional Neural Networks

2.2.1 Building Blocks of a Convolutional Neural Network

A Convolutional Neural Network (CNN) [1] is a type of artificial neural network designed for data with a grid-like structure, such as the pixel grid of an image. A CNN slides a small filter, or **kernel**, across the image, reusing the same weights at every position. This weight sharing keeps the number of learnable parameters low even for large images, and reflects a reasonable assumption about images of physical objects: a pattern that is useful for identifying, say, the rippled surface of a functioning aerator is just as useful whether it

(a) Example of a functional aerator characterised by its "leopard print" pattern.

(b) Example of a dysfunctional aerator with a smooth surface, indicating poor performance.



(c) Functional clarifier on the left, dysfunctional clarifier on the right. The functional clarifier has a clear surface, while the dysfunctional one has solids floating on top.

Figure 2.1: Visual comparison of functional and dysfunctional wastewater treatment components.

appears in the top-left or bottom-right corner of the image. This makes CNNs particularly well suited to the classification task in this project, where the relevant visual cues, such as the "leopard print" texture of a functional aerator or the colour and clarity of a clarifier's surface, can appear anywhere within an image and at different scales.

Formally, given an input feature map X with C_{in} channels and a kernel K of size $k \times k$, a convolutional layer computes an output feature map Y where each output value is a weighted sum of a small $k \times k$ neighbourhood of the input, taken across all input channels:

$$Y_{i,j,m} = b_m + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^{C_{in}} K_{u,v,c,m} \, X_{i+u,\, j+v,\, c} \tag{2.1}$$

where:

* i, j index the spatial position (row, column) of the output value being computed;
* m indexes the output channel, i.e. which of the C_{out} kernels produced this value;
* u, v index the position within the $k \times k$ kernel window, ranging over the neighbourhood being summed;
* c indexes the input channel being summed over;
* b_m is a learnable bias term, one per output channel.

In words, $Y_{i,j,m}$ is obtained by placing the kernel so its top-left corner is located at position (i, j) in X, multiplying it element-wise against every input channel underneath it, and summing the result. Each of the C_{out} kernels applied at a layer produces one such output

channel, also called a **feature map**, so a convolutional layer with C_{out} kernels turns a C_{in}-channel input into a C_{out}-channel output.

Because the same kernel weights are reused at every position (i, j), a single convolutional layer only ever looks at a $k \times k$ patch of its input. Depth is what lets the network see further: stacking layers causes the **receptive field** (the area of the original image able to influence a given output value) to grow, since a unit in the second layer combines several first-layer outputs, each of which already summarised a $k \times k$ patch of the original image. Two stacked $k \times k$ convolutions therefore give a receptive field of $(2k-1) \times (2k-1)$ pixels, even though neither individual kernel looks beyond its own $k \times k$ window. This is what allows a CNN to build up from simple, local features such as edges and colour gradients in its early layers to increasingly large-scale and abstract patterns, such as the overall texture or shape of a component, in its deeper layers.

Convolutions are usually applied with a **stride** s (the step size the kernel moves between applications) and zero-**padding** p (extra rows and columns of zeros added around the input border, used to maintain the input-output dimensions of the convolution). For a square input of dimensions $W \times W$, the resulting output dimension is

$$O = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1, \tag{2.2}$$

where O is the resulting width and height of the output feature map such that the output feature map has dimensions $O \times O$. A stride greater than one is a common way of downsampling a feature map directly within a convolutional layer, rather than relying solely on a separate pooling layer.

After each convolution, an element-wise, non-linear **activation function** is applied to every value of Y. This step is not optional: without a non-linearity, a stack of convolutions collapses mathematically into a single, larger linear filter, no matter how many layers are used, since a composition of linear operations is itself linear. All of the architectures used in this project use the Rectified Linear Unit (ReLU), or a smooth variant of it:

$$\mathrm{ReLU}(x) = \max(0, x), \tag{2.3}$$

which simply passes positive input values through unchanged while producing a zero for negative inputs.

Pooling layers are used to progressively reduce the spatial size of the feature maps, most commonly by taking the maximum value within each small window:

$$Y_{i,j} = \max_{0 \le u,v < k} X_{i \cdot s+u,\ j \cdot s+v}, \tag{2.4}$$

where i, j again index the output position, and u, v index the $k \times k$ pooling window

itself, now taken over a single channel at a time rather than summed across channels as in Equation 2.1. Applied with a stride s equal to the window size k, this reduces the resolution of the feature map, reducing the computation required by later layers while making the resulting representation somewhat more robust to small translations of the input.

By repeating convolutional, activation, and pooling layers, a CNN gradually trades spatial resolution for channel depth: early feature maps are large and shallow, and later ones are small and deep. Once the feature map is small enough, it is flattened, or globally pooled, into a single feature vector and passed through one or more fully-connected layers, ending in an output layer with one unit per class. A **softmax** function then converts these raw scores, or **logits** z_k, into a probability distribution over the K possible classes:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad k = 1, \ldots, K, \tag{2.5}$$

where z_k is the logit for class k, p_k is its resulting probability, and the sum in the denominator is over all K classes so that the resulting values of p_k sum to one. The classification decision is the class assigned the highest probability, and the network is trained by adjusting every learnable kernel and weight to make p_k as close as possible to 1 for the correct class, most commonly via the cross-entropy loss.

Training a CNN's convolutional layers entirely from scratch typically requires very large labelled datasets, since a randomly-initialised network has to learn everything from edge detection upward using only the available training images. This project instead relies on **transfer learning**: each architecture is first pretrained on ImageNet [2], a benchmark dataset of over a million labelled natural images spanning a thousand object classes, so that its convolutional layers already encode a broad range of general-purpose visual features before ever seeing a wastewater treatment image. Its final classification layer is then removed and replaced with a small classifier head fitted to the classes used in this project, and the network is fine-tuned on the far smaller WWTW image dataset described in Chapter 4. Six such pretrained architectures, all originally developed for the ImageNet benchmark, were evaluated in this project: **ResNet-18** and **ResNet-50** [3], **DenseNet-121** [4], **Inception-v3** [5], **EfficientNet-B0** [6], and **ConvNeXt-Tiny** [7]. Each combines the convolutional building blocks above in a different way, as described in the remainder of this section.

2.2.2 Residual Networks: ResNet-18 and ResNet-50

Residual Networks (ResNet) [3] address a specific problem for CNNs: gradients computed during backpropagation must pass back through every layer, and can shrink so much along the way that the earliest layers barely update at all (the **vanishing gradient** problem). Beyond a certain depth, plain networks made only of stacked convolutions begin

to perform worse, even on the training set, purely because they become harder to optimise, not because they lack capacity. ResNet's solution is the **residual**, or **skip**, connection: instead of asking a stack of convolutional layers to learn a full mapping $H(x)$ of its input x, the stack is made to learn only the residual difference $F(x) = H(x) - x$, and the original input is added back in:

$$y = F(x, \{W_i\}) + x. \tag{2.6}$$

Because the shortcut path carries x forward unchanged, a gradient flowing backward through the block always has a direct, unimpeded path to earlier layers, in addition to the path through F. This keeps even very deep residual networks trainable. The addition in Equation 2.6 requires $F(x)$ and x to share the same shape, so whenever a block changes the spatial resolution or number of channels, a 1×1 convolution is applied to the shortcut path to match its output shape.

ResNet-18 and ResNet-50 share the same overall structure, but differ in the type of block that structure is built from. ResNet-18 uses a **basic block** of two stacked 3×3 convolutions. ResNet-50 instead uses a deeper **bottleneck block**: a 1×1 convolution first reduces the number of channels, a 3×3 convolution operates on this reduced, cheaper representation, and a second 1×1 convolution restores the channel count before the shortcut is added. This keeps the computational cost of each block manageable even as the number of channels grows deep into the network. This makes it feasible for ResNet-50 to be both deeper and wider than ResNet-18. Table 2.1 summarises the resulting stage-by-stage structure of both networks for a 224×224 input image.

2.2.3 Densely Connected Networks: DenseNet-121

DenseNet [4] takes the shortcut-connection idea underlying ResNet a step further. Within a **dense block**, every layer receives the feature maps produced by *all* preceding layers in that block as its input, concatenated along the channel dimension, rather than only from the layer immediately before it:

$$x_l = H_l\Big([x_0, x_1, \ldots, x_{l-1}]\Big), \tag{2.7}$$

where $[x_0, x_1, \ldots, x_{l-1}]$ denotes the concatenation of the feature maps produced by layers 0 to $l-1$ along the channel dimension. H_l is a composite function including batch normalisation, ReLU, and a convolution. Because feature maps are concatenated rather than summed as in Equation 2.6, each layer only needs to contribute a small number of new feature maps, called the **growth rate** k, to the block's accumulated set; DenseNet-121 uses $k = 32$. If a dense block starts with k_0 input channels, layer l within that block therefore receives $k_0 + k(l-1)$ input channels, as calculated for the first dense block in Table 2.2. This lets a very deep network reuse features computed earlier in a block directly, instead of re-deriving them, and keeps each individual layer narrow, so the network can be

Table 2.1: Stage-by-stage structure of ResNet-18 and ResNet-50 for a 224×224 input. Bracketed blocks are repeated the stated number of times, with the layers inside each bracket applied in sequence. FC: fully-connected layer.

Stage	**Output size**	**ResNet-18**	**ResNet-50**
conv1	112×112	7×7, 64, stride 2	
pool	56×56	3×3 max pool, stride 2	
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
Global average pool + FC		$\rightarrow$ softmax over output classes	
Parameters		11.7M	25.6M

both deep and comparatively small in terms of parameter count. Because concatenation would otherwise let the channel count grow without bound, a **transition layer** is placed between dense blocks, using a 1×1 convolution to halve the number of channels and 2×2 average pooling to halve the spatial resolution.

DenseNet-121 stacks four dense blocks of 6, 12, 24, and 16 layers respectively, separated by three transition layers, after an initial 7×7 convolution and max pooling, and ending in a global average pooling and fully-connected classification layer, giving roughly 8.0M parameters in total. This corresponds to around a third of the number of parameters used by ResNet-50, despite being considerably deeper.

Table 2.2: Growth in the number of feature-map channels available to each layer within DenseNet-121's first dense block ($k_0 = 64$ input channels, growth rate $k = 32$).

Layer l	**Input channels**	**New channels**	**Next layer's input**
1	64	32	96
2	96	32	128
3	128	32	160
4	160	32	192
5	192	32	224
6	224	32	256

2.2.4 Inception Networks: Inception-v3

Rather than a single path of convolutions per stage, Inception-v3 [5] performs several convolutions of different kernel sizes in parallel on the same input feature map, and concatenates these outputs channel-wise into a single output. A given visual cue can appear at different scales in different images, so allowing receptive fields of different sizes side by side lets a layer use whichever scale is most informative, rather than committing to one fixed kernel size for the whole network.

Run naively, however, computing several full-size convolutions in parallel at every layer would be computationally expensive. Inception-v3 controls this cost with two factorisations. First, a larger $n \times n$ convolution is replaced with two smaller convolutions applied in sequence, e.g. a 5×5 convolution is replaced by two stacked 3×3 convolutions, since a stack of two 3×3 kernels spans the same area with fewer weights. Second, an $n \times n$ convolution can itself be factorised asymmetrically into a $1 \times n$ convolution followed by an $n \times 1$ convolution, reducing the cost further still for larger n. Each branch is also preceded by a cheap 1×1 convolution that reduces the number of channels before the more expensive spatial convolution is applied, in the same spirit as the ResNet-50 bottleneck described in Section 2.2.2. Table 2.3 summarises the branches of an Inception-v3 module.

Because Inception-v3 is a substantially deeper network than the others considered here, an **auxiliary classifier** is also attached partway through the network during training, computing its own cross-entropy loss from an intermediate feature map. The additional gradient signal this adds helps train the earlier layers of the network. The auxiliary branch plays no part during inference, and is discarded after training. Unusually among the architectures used in this project, Inception-v3's more aggressive early downsampling also requires a larger 299×299 input image, rather than the 224×224 used by the other architectures.

Table 2.3: Branches of a representative Inception-v3 module. Each branch is applied to the same input feature map, and their outputs are concatenated channel-wise to form the module's output.

Branch	Operations	Effective receptive field
1	1×1 conv	1×1
2	1×1 conv $\rightarrow$ 3×3 conv	3×3
3	1×1 conv $\rightarrow$ 3×3 conv $\rightarrow$ 3×3 conv	5×5 (factorised)
4	3×3 average pool $\rightarrow$ 1×1 conv	3×3
Output: channel-wise concatenation of branches 1–4		

2.2.5 Efficient Networks: EfficientNet-B0

The architectures described so far were all designed by hand, then made deeper. EfficientNet [6] instead starts from the observation that a network's depth (number of layers), width

(number of channels per layer), and input resolution should be scaled together, rather than increasing just one of them, and derives a small baseline network, EfficientNet-B0, using neural architecture search. EfficientNet-B0's building block is the **mobile inverted bottleneck convolution** (MBConv), composed of, in order: a 1×1 convolution that *expands* the number of channels by an expansion ratio t; a $k \times k$ **depthwise** convolution, which applies a single filter per input channel independently rather than combining all channels as in Equation 2.1; a **squeeze-and-excitation** block [8], which globally average-pools each channel and passes the result through two small fully-connected layers to produce a per-channel gate that reweights the feature map; and a final 1×1 convolution that *projects* the result back down to a smaller number of output channels. A residual connection, as in Equation 2.6, is added around the whole block whenever its input and output shapes match.

The depthwise convolution is the key source of MBConv's efficiency: applying a $k \times k$ convolution as a single per-channel filter followed by a 1×1 convolution to mix channels costs $k^2 C_{in} HW + C_{in} C_{out} HW$ multiply-adds, where H and W are the height and width of the feature map the convolution is applied to, versus $k^2 C_{in} C_{out} HW$ for an ordinary convolution that mixes channels and space at once, a saving of roughly a factor of C_{out} for typical kernel sizes. Table 2.4 gives the resulting stage-by-stage structure of EfficientNet-B0. Scaling this baseline network up to the larger EfficientNet-B1 through B7 variants, which are not used in this project, is done using a single **compound coefficient** ϕ that jointly scales depth, width, and resolution:

$$\text{depth} = \alpha^{\phi}, \qquad \text{width} = \beta^{\phi}, \qquad \text{resolution} = \gamma^{\phi}, \qquad \text{s.t. } \alpha\beta^2\gamma^2 \approx 2, \tag{2.8}$$

with $\alpha, \beta, \gamma \geq 1$ found by a small grid search, so that a single ϕ controls how much extra computation the network is allowed relative to the $\phi = 0$ baseline used in this project.

Table 2.4: Stage-by-stage structure of EfficientNet-B0 for a 224×224 input.

Stage	Operator	Resolution	Channels	Layers
1	Conv 3×3	224×224	32	1
2	MBConv1, 3×3	112×112	16	1
3	MBConv6, 3×3	112×112	24	2
4	MBConv6, 5×5	56×56	40	2
5	MBConv6, 3×3	28×28	80	3
6	MBConv6, 5×5	14×14	112	3
7	MBConv6, 5×5	14×14	192	4
8	MBConv6, 3×3	7×7	320	1
9	Conv 1×1 + pool + FC	7×7	1280	1

2.2.6 ConvNeXt-Tiny

ConvNeXt [7] takes the opposite approach to EfficientNet: rather than searching for a new architecture, it starts with a standard ResNet and modernises it one design choice at a time, borrowing several ideas from Vision Transformers. Table 2.5 contrasts the resulting ConvNeXt block with the ResNet bottleneck block described in Section 2.2.2.

The main structural changes are as follows. The convolution is enlarged from 3×3 to 7×7, but is made **depthwise** (as described in Section 2.2.5) to keep the added cost manageable. The block is also turned into an **inverted bottleneck**. In a ResNet-50 bottleneck block, the residual connection joins two wide ends and the 3×3 convolution works on a narrow middle: a block in the first stage, for example, reduces 256 channels to 64, convolves them, and expands them back to 256 (Table 2.1). The ConvNeXt block reverses this, so that its ends are narrow and its middle is wide: the first 1×1 convolution expands the channel count by a factor of 4 and the second projects it back down, so a block in ConvNeXt-Tiny's first stage expands 96 channels to 384 and projects them back to 96. This is the same expand-then-project structure as the MBConv block in Section 2.2.5, and the ConvNeXt authors note that it also matches the MLP block of a Transformer, whose hidden layer is four times wider than its input [7]. The depthwise convolution is then moved to the start of the block, before the expansion, so that the large 7×7 kernel operates on the narrow representation while the two 1×1 convolutions do most of the computation on the wide one. Batch normalisation is replaced with **layer normalisation**, which normalises across the channels of a single sample rather than across a batch of samples, making training less sensitive to batch size. ReLU is replaced with the smoother **GELU** activation (Gaussian Error Linear Unit):

$$\mathrm{GELU}(x) = x\,\Phi(x), \tag{2.9}$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution, that is, the probability that a standard normal random variable is less than x. ReLU passes positive inputs through unchanged and sets every negative input to exactly zero, so its output depends only on the input's sign. GELU instead multiplies each input by $\Phi(x)$, which rises smoothly from 0 to 1: large positive inputs pass through almost unchanged ($\Phi(x) \approx 1$), large negative inputs are pushed towards zero ($\Phi(x) \approx 0$), and inputs near zero are scaled by a factor in between, so small negative inputs produce small negative outputs rather than zero. Its authors describe this as weighting inputs by their value rather than gating them by their sign [9]. Unlike ReLU, which has a sharp corner at zero, GELU is smooth everywhere.

At the input of the network is the **stem**: the first layer, or small handful of layers, that processes the raw input image before it reaches the main stack of blocks. Because the input image is at full resolution, performing many channels of convolution would be prohibitively

expensive. The stem shrinks the image down quickly within the first few layers. ResNet's stem does this with a 7×7 convolution followed by a max pool. ConvNeXt replaces this with a single non-overlapping 4×4 convolution with stride 4, sometimes called a **patchify** layer since it behaves like the patch-embedding step at the start of a Vision Transformer: the image is cut into non-overlapping 4×4 tiles, and each tile is linearly projected into a single vector. Downsampling between the network's four stages is likewise handled by a separate 2×2 convolution with stride 2, rather than being folded into the first block of each stage as it is in ResNet.

ConvNeXt-Tiny arranges its blocks into four stages of 3, 3, 9, and 3 blocks, with channel counts of 96, 192, 384, and 768 respectively, giving roughly 28.6M parameters in total, comparable to ResNet-50.

Table 2.5: Operation-by-operation comparison of a ResNet bottleneck block and a ConvNeXt block. Both blocks add their input back onto their output via a residual connection.

Step	ResNet bottleneck block	ConvNeXt block
1	1×1 conv (reduce channels)	7×7 depthwise conv
2	Batch norm + ReLU	Layer norm
3	3×3 conv	1×1 conv (expand $\times 4$ channels)
4	Batch norm + ReLU	GELU
5	1×1 conv (expand channels)	1×1 conv (project channels)
6	Batch norm	—
Add input via residual connection, then (ResNet only) apply a final ReLU		

2.2.7 Summary of Architectures

Table 2.6 summarises the architectures used in this project. Besides the six pretrained architectures described above, it includes AlexNet [10], a much older and simpler network of five convolutional layers followed by three fully connected layers, which is trained from scratch in this project as a baseline (Section 5.2). The ImageNet column gives each architecture's size as defined in torchvision, with its original 1000-class output layer. The last two columns give the size of the final models trained in this project, after that output layer has been replaced with the tuned classifier head of each component (Section 5.5).

Two points stand out from Table 2.6. First, a network's parameter count says little about its age or depth: AlexNet, the oldest and shallowest network, is the largest, because about 58.6M of its 61.1M parameters sit in its three fully connected layers, while DenseNet-121, with 121 layers, is one of the smallest. Second, the classifier head can change a model's size considerably, since a single fully connected layer of 2048 neurons on top of a backbone with 2048 output features, as in ResNet-50, already adds about 4.2M parameters. EfficientNet-B0's aerobic zone model is the clearest example: its two-hidden-layer head

Table 2.6: Summary of the seven architectures used in this project, ordered by year of publication, with their parameter counts in millions. The clarifier and aerobic zone columns include the tuned classifier head of each final model. Inception-v3's totals include its auxiliary classifier, which is only used during training.

			Parameters		
Architecture	**Building block**	**Year**	**ImageNet**	**Clarifier**	**Aerobic**
AlexNet	Plain convolution	2012	61.1M	57.0M	57.0M
ResNet-18	Basic residual block	2016	11.7M	16.4M	12.0M
ResNet-50	Bottleneck block	2016	25.6M	31.9M	31.9M
Inception-v3	Inception module	2016	27.2M	32.7M	25.7M
DenseNet-121	Dense block	2017	8.0M	7.5M	13.3M
EfficientNet-B0	MBConv	2019	5.3M	4.0M	10.8M
ConvNeXt-Tiny	ConvNeXt block	2022	28.6M	29.4M	33.6M

of 2048 neurons adds about 6.8M parameters to a backbone of about 4.0M, more than doubling the size of the network.

2.3 Evaluation Metrics for Classification

A classifier assigns every test image to one class, and the assigned class is compared with the true class of the image. The metrics in this section summarise these comparisons over a test set.

Accuracy is the fraction of test images that are assigned to their true class. It is not suitable when the classes are imbalanced, since it is dominated by the common classes: a classifier that assigns every image to the majority class can reach a high accuracy while never identifying a rare class.

Precision, recall and F1 score are calculated for every class separately. With TP_c, FP_c and FN_c the number of true positives, false positives and false negatives for class c,

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \tag{2.10}$$

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{2.11}$$

Precision is the fraction of images assigned to class c that belong to it, and recall is the fraction of the images of class c that are assigned to it. The F1 score is the harmonic mean of the two, and is only high when both are high. The macro-averaged precision, recall and F1 score are the unweighted means of the per-class values, so that every class has the same weight regardless of its size.

The **confusion matrix** counts, for every true class, how many of its test images were assigned to each class. It shows which classes are mistaken for each other, in addition to how many images are misclassified.

The **receiver operating characteristic (ROC) curve** plots the true positive rate

(recall) against the false positive rate as the decision threshold on the output score of a classifier is varied. The **area under the ROC curve (AUC)** summarises this curve in a single number that does not depend on a specific threshold. An AUC of 1 means that every positive image receives a higher score than every negative image, while an AUC of 0.5 corresponds to random guessing. For more than two classes, a one-vs-rest ROC curve is calculated for every class from the softmax probability of that class, and the AUC values of the classes are averaged.

Loss and accuracy curves record the training and development loss and accuracy after every epoch. They are not used to compare models, but show whether training converged and when the model started to overfit the training set.

2.4 Gradient-Weighted Class Activation Mapping

Gradient-weighted class activation mapping (Grad-CAM) [11] produces a heatmap that shows which regions of an input image contributed the most to the prediction of a CNN for a given class.

Grad-CAM uses the output of the last convolutional layer, which is the layer before the global pooling and the fully connected classification layers. This output consists of one feature map per channel, and A^k denotes the feature map of channel k, with A^k_{ij} the element in row i and column j. A weight α^c_k is calculated for each channel by taking the gradient of the score y^c of class c, which is the logit of that class before the softmax (Equation 2.5), with respect to every element of A^k, and averaging these gradients over the feature map:

$$\alpha^c_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A^k_{ij}}, \tag{2.12}$$

where the sums run over all rows i and columns j of the feature map, and Z is the number of elements in the feature map (its height multiplied by its width).

The feature maps of all channels are then multiplied by their weights and summed, and only the positive contributions are kept:

$$L^c = \mathrm{ReLU}\left(\sum_k \alpha^c_k A^k \right). \tag{2.13}$$

The map L^c has the same height and width as the feature maps of the last convolutional layer, which are much smaller than the input image. It is therefore resized to the dimensions of the input image, normalised to the range $[0, 1]$, and superimposed on the image as a heatmap.

Literature Review

This chapter reviews the work that focuses on one question: can the state of an aerobic zone or clarifier be judged reliably from historical satellite images, without a site visit? No single field answers it. The relevant studies are spread over four areas, namely image analysis inside treatment plants, satellite remote sensing of water quality, remote sensing of treatment plants, and the behaviour of convolutional neural networks (CNNs) on small and uneven datasets.

3.1 The Monitoring Problem in South Africa

The 2022 Green Drop report for the Department of Public Works shows a clear gap between official records and actual plant conditions [12]. Out of 115 systems, 102 are in a critical state. The report notes a disconnect between administrative data and physical fieldwork. Physical inspections found differences of up to 59% between paper-based scores and the real state of the facilities. This project proposes using imagery to spot these differences, as the report indicates that scores based on submitted paperwork can hide true performance.

3.2 Reading Process State from Images Inside the Plant

Researchers have long tried to turn the operator's visual judgement of activated sludge into measurement. Quantitative image analysis of microscope images of flocs and filaments has grown into a field of its own [13], but each system needs hand-tuned steps such as thresholding, and a practitioner must choose which algorithms and threshold values to use [14]. CNNs remove much of this by learning features from labelled examples. Satoh et al. [15] trained binary CNN classifiers that recognised changes in floc morphology and the abundance of filamentous bacteria. Borzooei et al. [14] predicted the sludge volume index from microscope images taken weekly at a Belgian industrial plant over almost two years. Transfer learning from ImageNet generally offer better performance than training from scratch. Large networks can overfit, and the small ConvNeXt-nano was seen to generalise best ($R^2 = 0.75$). The model of Borzooei et al. [14] also underestimated the highest index values, which the authors attribute to a very small number of such samples in the training set.

In contrast to microscope, a camera above the tank can also be used. Carballo Mato et al. [16] used images from a pan-tilt-zoom camera at a working plant to segment surface foam with texture segmentation models, reaching a Dice score of 86%, and reported the percentage of the tank surface covered by foam to the operators in real time.

These studies agree that the appearance of sludge and tank surfaces carries real information about process state, and that CNNs can extract it from modest datasets. This supports the premise of this project. The studies by Borzooei et al. [14] and Carballo Mato et al. [16], however, both need sampling or equipment at the plant, which is the cost the problem statement wants to avoid. Borzooei et al. [14] also trained and tested their models on samples from a single plant, so their results do not show whether a model can judge a plant it has not seen.

3.3 Satellite Remote Sensing of Water Quality

Satellite studies of water quality focus on suspended solids and turbidity, because water clarity falls as the suspended solids concentration rises [17]. A clarifier that eliminates solids should therefore look brighter than one that retains them. At close range a CNN can detect suspended solids in ordinary colour photographs. Lopez-Betancur et al. [18] fine-tuned a pretrained AlexNet on smartphone images of liquid samples and estimated total suspended solids with 98% accuracy, although under controlled LED lighting.

The main challenge to applying these satellite methods to treatment plants is image resolution. Satellite studies estimate suspended solids and turbidity by analysing the light reflected off the water surface, but they typically use medium-resolution satellites such as Landsat [17], whose pixels are tens of metres wide. This is too coarse for small bodies of water. A clarifier would be resolved as only a few pixels, and the surface pattern of an aeration zone would not be discernible. Google Earth imagery provides more detail. In remote sensing it is increasingly used to build reference data for land cover mapping through visual interpretation, where an analyst labels what is seen in the image instead of computing it from the reflected light [19]. Its coverage is uneven, however. Lesiv et al. [19] mapped where and when such imagery exists worldwide and found that availability is biased towards regions such as the USA, Europe and India, with clear breaks at political borders. Hu et al. [20] compared Google Earth imagery for land cover mapping with imagery from QuickBird, a commercial satellite that captured sub-metre images in four spectral bands [21]. Google Earth was almost as accurate. It did well on classes that can be recognised by shape and context, but poorly on classes that can only be separated by colour, because it carries little spectral information. The classes in this project are of the second kind. Every clarifier is round and every aeration zone is rectangular, so the classes differ only in surface appearance. The limited spectral information also means that a physical quantity such as suspended solids cannot be computed from the imagery. The project must therefore learn the visual categories that operators already use.

3.4 Remote Sensing of Treatment Plants

In the remote sensing literature, treatment plants appear mainly as objects to be found. Wastewater treatment plant is one of 38 scene classes in the PatternNet benchmark, with one label per plant whatever its condition [22]. Li et al. [23] combined OpenStreetMap labels with multimodal satellite data to detect plants and tested the method around Stuttgart. Shuai et al. [24] built an attention-based detector for Gaofen-2 imagery that located plants across Beijing with 80.1% accuracy at 90.4% recall, and noted that the mix of rectangular and circular tanks makes plants harder to detect with conventional methods. Randhawa et al. [25] worked towards worldwide detection. In all of this work the tanks help to find a plant, and none of these studies asks whether the tanks are working. The closest regulatory precedent, from agriculture, has the same limit. Handan-Nader and Ho [26] applied a CNN to high-resolution imagery of North Carolina, found 589 more poultry feeding operations than a manual count had (an increase of 15%), and argued that the method can inform compliance priorities. Their target was the location of a facility, not its behaviour.

3.5 Pretrained CNNs on a Small, Texture-Defined Problem

With little labelled data, training a CNN from scratch is usually not practical. Nogueira et al. [27] compared full training, fine-tuning and fixed feature extraction for six CNNs on three remote sensing datasets and found that fine-tuning tended to work best, a result that Borzooei et al. [14] confirm in a very different domain. This project will therefore focus on fine-tuning pretrained networks, and train onl yone model (AlexNet) from scratch as a baseline (Section 5.2). The earlier study by Penatti et al. [28] adds a note of caution. ImageNet features gave the best results on aerial scenes but were beaten by a simple colour descriptor on multispectral scenes of coffee crops. Features learnt from everyday objects may therefore miss a cue when classes differ mainly in colour, as the clarifier classes do.

Texture is the other cue. Geirhos et al. [29] showed that ImageNet-trained CNNs rely on texture far more than people do, which they framed as a weakness for recognising objects. In this project, on the other hand the bias may help, since a working aeration zone is identified by its surface texture, and Carballo Mato et al. [16] likewise treated tank foam as a texture problem. However, concrete walls, walkways and the rotating arms of clarifiers also have strong textures that a model may learn in place of the water. The six pretrained models compared in this project come from five families published between 2016 and 2022 [3–7], and no comparison of these families on treatment plant or similar imagery was found in the literature. Since Borzooei et al. [14] found that larger models did worse on limited data, the smaller variants were prioritised.

Uneven class distributions adds a further problem, because the rare states may be the ones that are most important to find. Buda et al. [30] showed that class imbalance harms

CNN performance and that oversampling was the best remedy in almost all their tests. Lin et al. [31] proposed focal loss, which down-weights easy examples so that training focuses on hard ones. Both methods were developed on data with distinct classes, such as handwritten digits and everyday objects. The classes here are graded judgements, defined by thresholds such as a third or two thirds of the surface, so a hard example may be borderline or mislabelled rather than informative. Neither study examined that case. This project uses neither method, and instead weights the cross-entropy loss of each class inversely to its frequency in the training set (Section 5.4).

3.6 Testing on Unseen Plants and Checking What the Model Sees

Kattenborn et al. [32] demonstrated that how you select test data is critical in remote sensing. When analysing drone imagery, choosing random test images made the model appear up to 28% more accurate than it actually was. This happens because images taken from the same area and at the same time are too similar, giving the model an unfair advantage. To get a realistic performance score, the model must be tested on entirely separate geographic locations, a rule that becomes even more important when you only have a small amount of data. The data in this project require this to be considered. Each tank appears many times at different dates, so a random split would put images of the same tank, taken months apart, into both the training and test sets.

Inflated scores are not the only risk. Geirhos et al. [33] describe shortcut learning, in which a network finds a rule that works on the test set but fails under new conditions, and Lapuschkin et al. [34] give the well-known case of a classifier that recognised horses by a copyright tag common in horse photographs. With a handful of plants imaged over many years, the equivalent shortcut would be to recognise which plant is in view and how often that plant was labelled as failing. Holding out whole facilities blocks this route. Checking where the model looks is a second safeguard. For example gradient-weighted class activation mapping (Grad-CAM) highlights the image regions that are important for a CNN's prediction [11], so it can test whether the model uses the same cues a human operator woud use, and not merely whether it reaches the same answer.

What to measure is the last open question. Overall accuracy hides poor results on rare classes, which is why Buda et al. [30] used a multi-class version of the area under the receiver operating characteristic (ROC) curve. Saito and Rehmsmeier [35] argue that precision-recall curves are more informative than ROC curves when classes are imbalanced, so the ROC-based AUC used in this project is reported alongside the per-class precision and recall rather than on its own. Cost matters as well. A tool that triggers inspections can accept some false alarms but should rarely miss a failing unit, so recall on the failing classes, at a threshold chosen for that purpose, is the more useful measure here (Section 5.9).

3.7 Summary and Conclusions

This chapter has reviewed the literature relevant to judging the state of aerobic zones and clarifiers from historical satellite images. Image analysis inside treatment plants shows that the appearance of sludge and tank surfaces carries information about process state, but these methods require sampling or equipment at the plant. Satellite studies of water quality use medium-resolution spectral data, which is too coarse for individual tanks, while Google Earth imagery has sufficient resolution but little spectral information. Remote sensing studies of treatment plants address the detection of plants and not their operating state, and no study was found that classifies the state of treatment units from satellite imagery. The literature further supports fine-tuning pretrained CNNs on small datasets, testing on facilities withheld from training, and reporting per-class precision and recall. In the next chapter we will describe the dataset of satellite images used in this project, its annotation and how it was split for training and testing.

Dataset

The dataset used in this project consists of satellite imagery sourced from Google Earth and covers five waste water treatment works (WWTW) facilities: Atlantis, Cape Flats, Waterval, Fisantekraal and Northern Works. Each image is a snapshot of a facility's biological reactors at a particular point in time, and its filename encodes the month and year in which it was captured, although the exact format differs between facilities (for example `Atlantis 01_2019`, `capeflats1_201408` and `Northern Works 2020 05`).

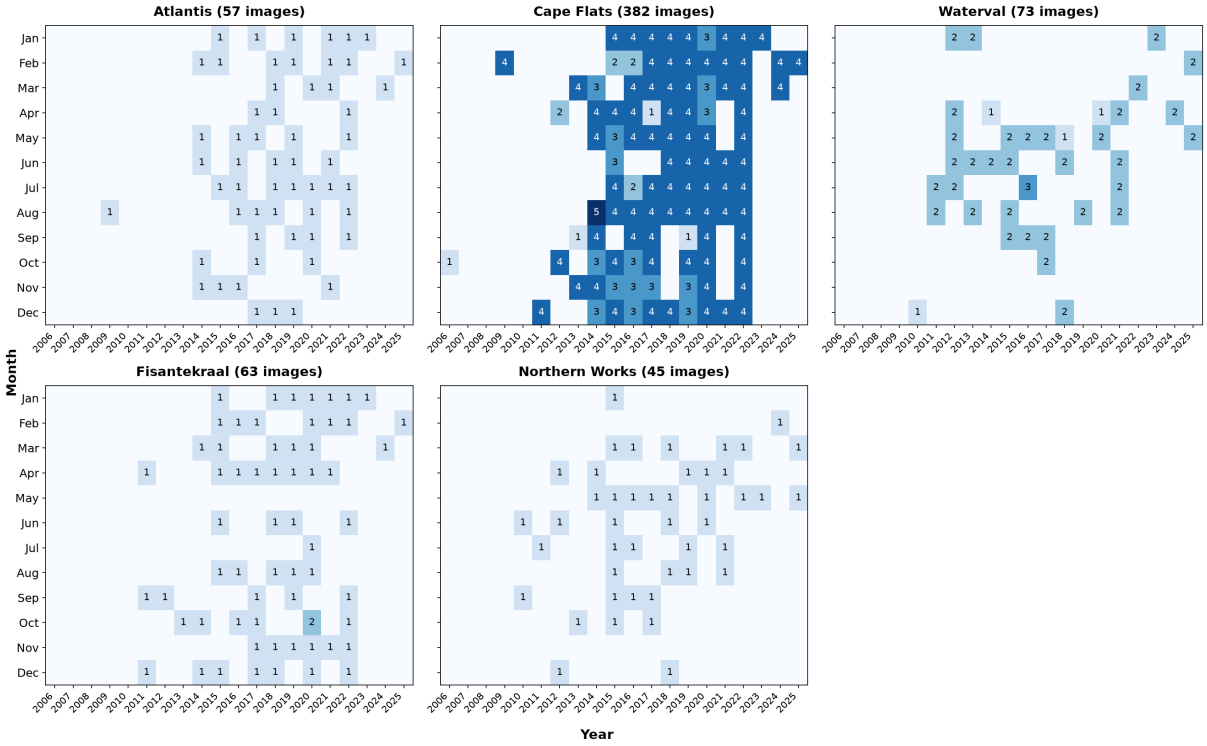


Figure 4.1: Number of labelled images per month for each of the five facilities. Cape Flats and Waterval have a separate image for each unit on most dates, so their cells can be larger than one.

The dataset contains 620 labelled images, captured between October 2006 and May 2025. As seen in Figure 4.1, most of them (423 of the 620) were captured between 2016 and 2022, and they are spread fairly evenly over the months of the year, with between 38 (November) and 62 (August) images per calendar month.

4.1 WWTW Sites

Atlantis WWTW is located on Mission Express Way in Cape Farms, Cape Town. As part of the greater Cape Town municipality, this facility falls within the Western Cape

province. Cape Flats WWTW is also situated in the Western Cape; this sewage treatment plant is located on Informal Road in the Southern Suburbs of Cape Town. Additionally, the Western Cape is home to Fisantekraal WWTW, which is situated approximately 10 kilometres north of Durbanville. Waterval WWTW is operated by ERWAT (Ekurhuleni Water Care Company) and is situated in the Gauteng province. Also located in Gauteng is Northern Works (Noordelike Werke) in Johannesburg. It is the largest wastewater treatment plant in Johannesburg and ranks among the largest in South Africa, treating approximately 400 million litres of wastewater every day. The locations of these sites are shown in Figure 4.2. An overview of the Atlantis, Cape Flats, Waterval, Fisantekraal, and Northern Works WWTW facilities is shown in Figure 4.3.

Figure 4.2: Location of the five WWTW facilities in the dataset.

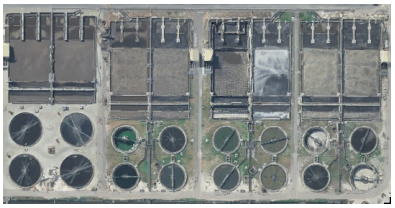
4.2 Annotations

Images were manually annotated using the VGG Image Annotator (VIA2) tool. Aerobic zones were labelled using rectangular bounding boxes, while clarifiers were labelled using circular regions, reflecting the physical shape of each structure. Each annotation carries a region attribute indicating the visible state of that component at the time the image was captured.

Aerobic zones were labelled into one of three categories, based on how much of the surface exhibits the characteristic "leopard print" aeration pattern. Examples are shown in Figure 4.4:

- **Functional**: the surface shows the leopard print pattern.
- **Suboptimal**: a third or more of the surface shows no leopard print, or the state is unclear.
- **Dysfunctional**: two thirds or more of the surface shows no leopard print, or is clearly dysfunctional.

(a) Atlantis



(b) Cape Flats

(c) Waterval

(d) Fisantekraal

(e) Northern Works

Figure 4.3: Overview of the Atlantis, Cape Flats, Waterval, Fisantekraal, and Northern Works WWTW facilities.

Clarifiers were labelled into one of four categories, based on surface colour and visibility. Examples are shown in Figure 4.5:

- **Functional**: surface appears uniformly black.
- **Dysfunctional**: surface appears green.
- **Scum**: surface is not uniformly black.
- **Empty**: shadows are visible inside the tank.

Annotations are stored as VIA2 project files in JSON format. Each entry records the image filename, file size, and a list of regions, where each region specifies its shape (rectangle or circle), pixel coordinates, and the associated class label. An example entry for an aerobic zone annotation is shown below:

```
"Atlantis 01_2019.jpg3469910": {
  "filename": "Atlantis 01_2019.jpg",
  "size": 3469910,
  "regions": [
    {
```

```
      "shape_attributes": {
        "name": "rect",
        "x": 2707, "y": 931,
        "width": 2839, "height": 1381
      },
      "region_attributes": { "aerobic zone": "Dysfunctional" }
    }
  ],
  "file_attributes": {}
}
```

Examples of the classes for aerobic zones and clarifiers are shown in Figures 4.4 and 4.5 respectively.

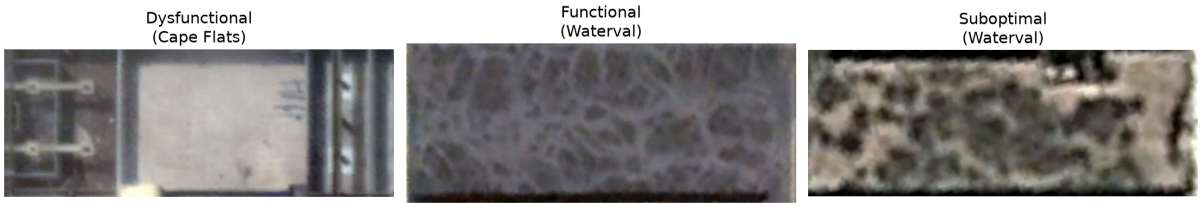


Figure 4.4: One example of each aerobic zone class, with the facility it comes from. Each example is the test image of the stratified split (Chapter 5) that all six pretrained architectures classified correctly with the highest mean confidence, so that it is a clear case of its class.

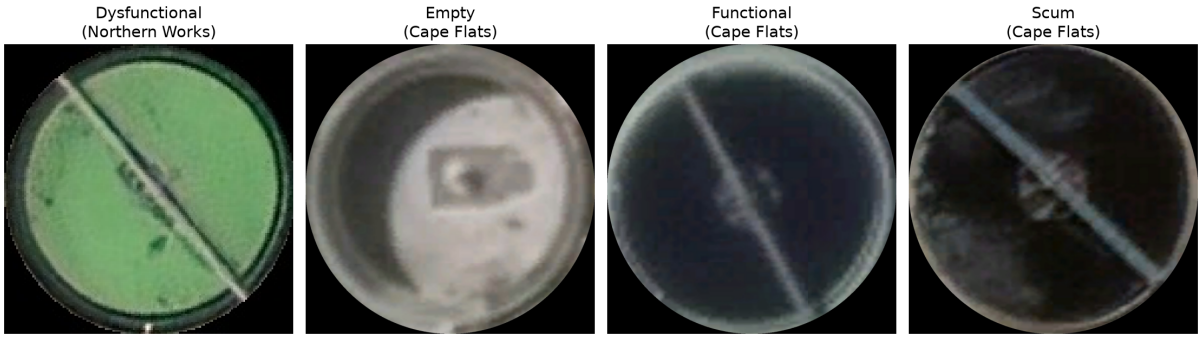


Figure 4.5: One example of each clarifier class, with the facility it comes from, chosen in the same way as in Figure 4.4. The area outside each tank is masked to black by the preparation script (Section 4.4).

4.3 Facility Structure

The five facilities differ slightly in how their data is organised, reflecting differences in plant layout.

Atlantis consists of a single biological reactor, so its 59 images are stored in one folder with no subdivisions. Two of these images have no labels, so 57 are part of the dataset. Aerobic zone and clarifier labels are each stored in a single JSON file.

Cape Flats consists of four separate reactor units (CF1–CF4), each with its own image folder and containing between 92 and 100 images. Each unit has its own aerobic zone and clarifier label file.

Waterval consists of two reactor units, referred to as Top and Bottom, with 37 and 36 images respectively. Each unit has its own aerobic zone and clarifier label file.

Fisantekraal consists of a single reactor unit, with 63 images stored in one folder. Only clarifiers have been labelled and stored in a single JSON file.

Northern Works consists of a single reactor unit, with 45 images stored in one folder. Only clarifiers have been labelled and stored in a single JSON file.

Table 4.1 summarises the structure of the dataset across the five facilities.

Table 4.1: Number of units and labelled images per facility.

Facility	**Units**	**Images**
Atlantis	1	57
Cape Flats	4	382 (96+94+92+100)
Waterval	2	73 (37+36)
Fisantekraal	1	63
Northern Works	1	45
Total	9	620

This structural difference between facilities, with a single unit at Atlantis, Fisantekraal and Northern Works, four at Cape Flats, and two at Waterval, is factored into the data pipeline described in Section 4.4, since each facility's images and labels need to be parsed slightly differently before they can be fed into the CNN.

4.4 Data Preparation

Before the labelled regions can be used to train a classifier, they need to be extracted from the full facility images and organised by class. This is handled by a preparation script (`prep.py`), which reads the VIA2 annotation files, crops out each labelled region, and saves the crops into a folder structure that can be loaded directly with `torchvision.datasets.ImageFolder`.

The script processes each facility one unit at a time, since the facilities have different numbers of units and label files. For every unit, it opens the aerobic zone and clarifier JSON files separately and works through each image entry in turn.

For each labelled region in an image, the script performs the following steps:

* Reads the shape attributes of the region. Aerobic zones are stored as rectangles, defined by an x and y coordinate along with a width and height. Clarifiers are stored as circles, defined by a centre point and a radius.
* Converts the shape into a bounding box that can be used to crop the image, since Python's Image Library crop function requires a rectangular box even for the circular clarifier regions.
* Clamps the box to the edges of the image, in case a region was drawn slightly outside the image bounds during annotation.
* Reads the class label from the region attributes and checks it against the expected set of classes for that component. Labels are also normalised, so that small inconsistencies in capitalisation from the manual annotation process do not create extra classes.
* For a clarifier, sets every pixel outside the annotated circle to black, so that only the tank itself remains visible. A rectangular aerobic zone fills its whole bounding box, so nothing is masked for the aerobic zones.
* Crops the region out of the source image and saves it as a JPG file inside the folder matching its class label.

Each output filename encodes the facility, unit, and original image name, so that a crop can always be traced back to the source image it came from. For example, a crop from unit CF2 at Cape Flats would be saved with a filename beginning `CapeFlats_Images-CF2_...`, followed by the original image name and a region number.

The result is a folder structure with one top-level folder per component and one subfolder per class:

```
cnn_dataset/
    aerobic_zone/
        Dysfunctional/
        Functional/
        Suboptimal/
    clarifier/
        Dysfunctional/
        Empty/
        Functional/
        Scum/
```

The two classes that are excluded from the dataset (Section 4.5) have no folder. This structure means the aerobic zone and clarifier sub-images are treated as two separate classification problems, each with its own set of class folders, rather than a single combined dataset.

4.5 Excluded from the dataset

During processing, two annotated classes were found to have too few examples to be usable for training. The aerobic zone dataset contained a single region labelled **Empty**, and the clarifier dataset contained four regions labelled **Stagnant**. Both classes were therefore excluded from the final dataset, and the corresponding regions were not cropped or saved. A class with only one or a handful of examples cannot be meaningfully split into training, development, and test sets, and would not give the CNN enough examples to learn a reliable representation of that class. Including such a small class also risks skewing the overall class balance and encouraging the model to simply ignore it during training, since the loss contribution from so few examples is negligible compared to the other classes. The excluded regions were still counted separately by the preparation script, so their presence in the original annotations is not lost, but they play no further part in the classification task.

4.6 Data Splitting

In the following section the way the available data is split into training, development and testing partitions is described.

4.6.1 Leave-one-out Split

To evaluate how well the trained classifier generalises to WWTW facilities it has never encountered, the dataset was partitioned using a leave-one-out strategy rather than a random split. One facility, chosen from the five available (Atlantis, Cape Flats, Waterval, Fisantekraal, and Northern Works), was the test facility and withheld from training entirely: none of its images appear in the training or development sets, and it plays no part in training or in selecting the checkpoint that is kept. The hyperparameters, which were tuned once on a single fold, are the one exception, as discussed in Section 5.8. Note that this is repeated once for each facility: five times for the clarifiers, and three times for the aerobic zones, which are only labelled at Atlantis, Cape Flats and Waterval. This guarantees that test performance reflects the model's ability to classify components at an unseen site.

This split was applied independently for each facility component, aerobic zone and clarifier, since the two have different classes and different image sets.

Once the test facility was removed, the remaining four facilities' images were divided into training and development sets. A random split at this stage risks an uneven mix: the development set could end up dominated by whichever facility or unit happened to contribute the most images, or by images from a narrow range of capture dates, which would give an unrepresentative estimate of development performance.

To avoid this, images were first grouped by facility, unit and class (some facilities

have more than one unit, for example Cape Flats has 4 units each with its own set of images). Different units at the same facility can differ in lighting, or capture date, so treating them as separate groups during shuffling keeps that variation spread across both training and development rather than concentrated in one. Within each group, images were shuffled using a fixed random seed to mix capture dates before being partitioned according to a fixed development ratio (20%). Splitting within these smaller groups rather than across the whole set ensures that every facility, unit, and class combination among the four training facilities is represented in both the training and development sets, in proportion to how much data it contributes. Figure 4.6 shows the resulting split for the clarifiers when Waterval is withheld as the test facility.

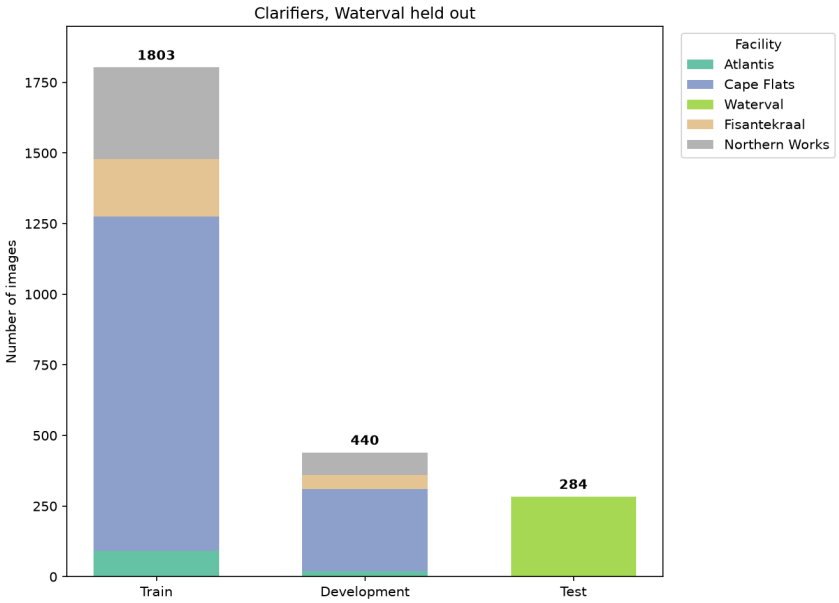


Figure 4.6: Number of clarifier images from each facility in the training, development and test sets when Waterval is held out. The test set contains only Waterval, while the remaining four facilities are split into training and development sets.

4.6.2 Stratified Train/Development Split

Before adopting the leave-one-out approach described above, a stratified split was used to produce training, development, and test sets directly from the dataset, without holding any facility out. This split follows a 70/20/10 training/development/test ratio and uses the same facility, unit, class grouping and shuffling procedure described in the previous section: images within each group are shuffled using a fixed random seed before being partitioned, so that every facility, unit, and class combination is represented across all three sets in proportion to how much data it contributes, and no single facility or unit dominates any one split.

4.7 Relabelling

The dataset used in this project is completely new, and was prepared shortly before the project started by annotators under the supervision of Prof F. de Wet. Inspection of the data in the early phases of experimental design however identified the occasional occurrence of labelling inconsistencies or errors. To improve label quality, a relabelling notebook was built using Jupyter widgets to review the clarifier dataset in full and correct labels where necessary. Every clarifier image was reviewed manually, rather than considering only images the classifier identified incorrectly which could potentially introduce bias. For each image, the notebook displays the image alongside its current label and a row of per-class buttons; clicking a button records the corrected label without requiring any manual file editing. Corrections are written to a separate `relabel/` working copy of the annotation files rather than overwriting the originals, so the original VIA2 labels remain intact and every correction can be traced back to the annotations as originally supplied.

Figure 4.7 shows a clarifier at Cape Flats that was originally labelled **Scum** and was corrected to **Empty** during the review, a correction the supervisor later accepted. The tank appears to have been drained: its floor is visible, and the dark band across it is the shadow of the bridge rather than scum on the water surface. Figure 4.8 shows the review interface itself, with the original label displayed in yellow next to the image and the correction buttons below it.

Figure 4.7: A relabelled clarifier from Cape Flats, shown with some of its surroundings. The original label was “Scum”, but the label was corrected to “Empty”, since the tank appears to be drained, with its floor and the shadow of the bridge visible inside it.

The review proposed 293 corrections in total. Each proposed correction was then checked by the project supervisor, who could accept it, reject it and keep the original label, or assign a different label. Of the 293 proposed corrections, 166 were accepted, 99 were rejected, and for 28 the supervisor assigned a different label, so 194 regions ended up with a label different from the original annotation. These verdicts were applied to the `relabel/` working copy with a script that first checks that every region still holds the

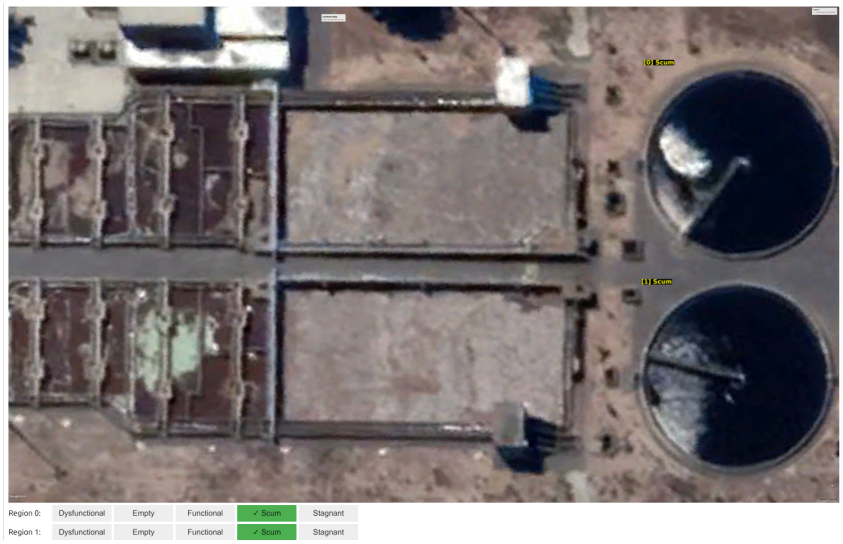


Figure 4.8: The relabelling interface, showing the original label and the per-class correction buttons.

proposed label, and that records every decision in a separate file. The resulting labels are the ones used in every experiment in Chapter 5.

Table 4.2 summarises the effect of relabelling on a per-class basis, broken down by facility. For each class, two columns are given: **Old**, the total number of clarifier regions at that facility carrying that label in the original annotations, and **New**, the total number carrying that label after relabelling and the supervisor's review. The rightmost **Relabels** column gives the number of regions at that facility whose final label differs from the original one.

Table 4.2: Total number of clarifier regions per class, per facility, before and after relabelling. "Old" is the count in the original annotations; "New" is the count after relabelling and the supervisor's review. "Relabels" is the number of regions at that facility whose final label differs from the original one.

	Dysfunctional		**Empty**		**Functional**		**Scum**		
Facility	Old	New	Old	New	Old	New	Old	New	**Relabels**
Atlantis	0	0	2	3	44	44	64	63	2
Cape Flats	151	137	113	120	696	679	516	540	68
Waterval	78	7	0	0	95	89	111	188	80
Fisantekraal	19	17	9	9	106	110	118	116	8
Northern Works	127	131	13	13	113	81	152	180	36
TOTAL	**375**	**292**	**137**	**145**	**1054**	**1003**	**961**	**1087**	**194**

4.8 Final Dataset

Tables 4.3 and 4.4 give the final number of images per class, per unit, after relabelling. These are the counts used in every experiment. The aerobic zone dataset contains 1743 images, all from Atlantis, Cape Flats and Waterval, and the clarifier dataset contains 2527 images from all five facilities. Both datasets are imbalanced. Suboptimal is the smallest aerobic zone class with 223 images, against 813 for Dysfunctional. Among the clarifiers, Empty has only 145 images, against 1087 for Scum. The classes are also spread unevenly across facilities: Waterval Top accounts for 344 of the 707 Functional aerobic zones, and Northern Works accounts for 131 of the 292 Dysfunctional clarifiers. Section 5.4 describes how this imbalance is handled during training.

Table 4.3: Summary of image counts per class for the Aerobic Zones.

Facility	Dysfunctional	Functional	Suboptimal
Atlantis	22	72	19
Cape Flats 1	113	44	37
Cape Flats 2	146	21	21
Cape Flats 3	137	15	32
Cape Flats 4	166	12	22
Waterval Top	85	344	15
Waterval Bottom	144	199	77
Fisantekraal	0	0	0
Northern Works	0	0	0
TOTAL	**813**	**707**	**223**

Note that Fisantekraal and Northern Works do not have any aerobic zones labelled.

Table 4.4: Summary of image counts per class for the Clarifiers.

Facility	Dysfunctional	Empty	Functional	Scum
Atlantis	0	3	44	63
Cape Flats 1	31	59	172	82
Cape Flats 2	58	35	176	95
Cape Flats 3	21	22	201	124
Cape Flats 4	27	4	130	239
Waterval Top	0	0	62	78
Waterval Bottom	7	0	27	110
Fisantekraal	17	9	110	116
Northern Works	131	13	81	180
TOTAL	**292**	**145**	**1003**	**1087**

Experimental Design

5.1 Overview

This chapter describes how the classification system was designed and configured, and which experiments were run with it. The clarifiers and the aerobic zones are treated as two separate classification problems (Chapter 4), but both are handled by the same system, and every experiment is carried out for both components unless stated otherwise. The first part covers the parts of the system that are shared by every experiment: the model architectures and how transfer learning is applied to them (Section 5.2), the augmentations applied to the training images (Section 5.3), how the class imbalance in the dataset is handled (Section 5.4), how the hyperparameters of each architecture were chosen (Section 5.5), and the training loop itself (Section 5.6).

The second part describes the three experiments that were carried out using this system:

1. **Mixed-facility baseline** (Section 5.7): the images of all five facilities are pooled and divided using the stratified 70/20/10 training/development/test split described in Chapter 4.

2. **Leave-one-facility-out evaluation** (Section 5.8): each facility is held out in turn as an unseen test facility, while the model is trained on the remaining facilities, with part of them held out as a development set. This gives five folds for the clarifiers, and three for the aerobic zones, which are only labelled at three facilities. This experiment estimates how well a model would perform on a facility it has never seen before.

3. **Binary (OK / Not-OK) approach** (Section 5.9): the classes are collapsed into a two-class problem, where **Functional** is treated as OK and every other class is treated as Not-OK: **Dysfunctional**, **Empty** and **Scum** for the clarifiers, and **Suboptimal** and **Dysfunctional** for the aerobic zones.

The chapter ends with the metrics used to evaluate every run (Section 5.10) and the Grad-CAM method used to inspect which parts of an image a model bases its prediction on (Section 5.11).

Figure 5.1 summarises the pipeline from start to finish. It starts from the labelled satellite images described in Chapter 4, which `prep.py` crops into individual images sorted into one folder per class. These images are then divided into training, development and test

sets, either with the stratified split of Experiment 1 or by facility, so that no image from the held-out test facility is ever used during training. Every image is resized and normalised before it is passed to the network, and images in the training set are also randomly augmented (Section 5.3). The network is one of the seven architectures in Section 5.2, with a classifier head sized by the hyperparameters chosen for that architecture (Section 5.5), and is trained using the loop described in Section 5.6.

The results of every run, meaning one architecture trained for one component on one split, are saved to a separate `results_*.pkl` file (Section 5.8). Each file contains the test metrics, the confusion matrix, the training history and the classification made for every test image.

5.2 Model Architectures and Transfer Learning

Seven CNN architectures are compared in this project: ResNet-18 and ResNet-50 (Section 2.2.2), DenseNet-121 (Section 2.2.3), Inception-v3 (Section 2.2.4), EfficientNet-B0

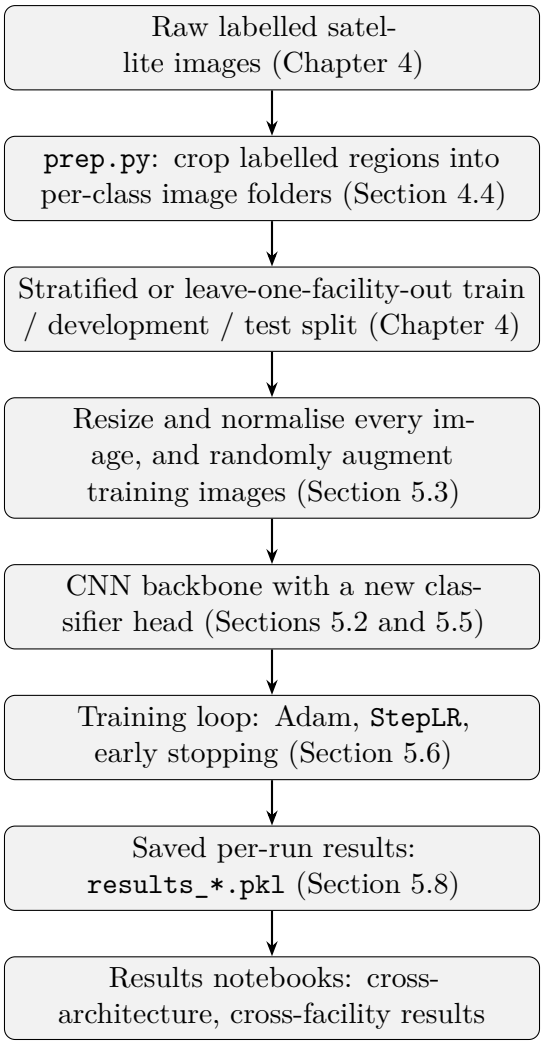


Figure 5.1: End-to-end experimental pipeline, from labelled satellite imagery to a saved, comparable results file per combination of component, architecture and test set.

(Section 2.2.5), ConvNeXt-Tiny (Section 2.2.6) and AlexNet. Table 5.1 lists their sizes, which range from 5.3 to 61.1 million parameters. All seven are taken from the torchvision library, and the same seven architectures are trained for both the clarifiers and the aerobic zones.

AlexNet is deliberately different from the other six: rather than being fine-tuned from ImageNet weights, it is trained entirely from randomly initialised weights, with its own separate, hand-picked configuration (Section 5.5). It was added to the project specifically to measure the effect of pretraining, that is, whether the far larger pretrained architectures outperform a classical network trained from scratch on this project's own, small dataset.

Table 5.1: Parameter count of each of the seven architectures evaluated, from smallest to largest, as defined in torchvision with the original 1000-class ImageNet classifier. The new classifier heads (Section 5.5) change these totals.

Architecture	Parameters
EfficientNet-B0	5.3M
DenseNet-121	8.0M
ResNet-18	11.7M
ResNet-50	25.6M
Inception-v3	27.2M
ConvNeXt-Tiny	28.6M
AlexNet	61.1M

For each of the six pretrained architectures, transfer learning follows the same recipe. The architecture's convolutional backbone is loaded with weights pretrained on ImageNet [2], specifically torchvision's `IMAGENET1K_V1` weights, except for ResNet-50, which uses the newer, more accurate `IMAGENET1K_V2` recipe. Its original 1000-class ImageNet output layer is then discarded, and a new classifier head, sized by that architecture's own tuned hidden layers (H) and neurons (N) (Section 5.5), is put in its place: a stack of H `Linear+ReLU` pairs followed by a final `Linear` layer mapping to the project's number of classes. Every parameter in the network, backbone included, is then fine-tuned on the WWTW dataset. No layers are frozen, so pretraining only supplies a starting point for every weight rather than fixing any of them in place.

Every one of these architectures ends in the same kind of structure: a convolutional backbone feeding into one or more fully connected layers that produce the 1000 ImageNet class scores. Replacing this final part therefore means locating whichever attribute of the model object torchvision happens to store it under, which differs from one architecture's implementation to the next. For ResNet-18, ResNet-50 and Inception-v3, it is a single attribute named `model.fc`. DenseNet-121 stores the same kind of single layer under `model.classifier` instead, which is simply a difference in torchvision's naming convention rather than an architectural difference. For EfficientNet-B0, ConvNeXt-Tiny and AlexNet,

`model.classifier` is itself a short stack of several layers rather than a single one, so only the specific layer within it that produces the final class scores, the last `Linear` layer, is replaced, and everything before it is left as it was. In practice, this means EfficientNet-B0 and AlexNet keep the dropout layer that is located just before the original output layer, and AlexNet also keeps its two existing hidden fully connected layers of 4096 neurons each, since these are simply the layers that fed into the one that got replaced.

Two architectures need one further, architecture-specific adjustment. Inception-v3's auxiliary classifier (Section 2.2.4), used only during training, also has its own small output layer resized to the project's number of classes. Unlike the main classifier head, it is always left as a single `Linear` layer regardless of the tuned hidden layers (H) and nuerons (N), since it is only a secondary training signal rather than the model's final output. ConvNeXt-Tiny's global average pooling layer is replaced with global max pooling, so the feature vector reaching its classifier head is composed of each channel's strongest activation rather than its average.

Every image is resized to a fixed input size before it is passed to the network. The clarifier crops are close to square, since they are cut around a circular tank, and are resized to 224×224 pixels. The aerobic zones are wide rectangles, with a median width of about 2.6 times their height, and are resized to 112×224 pixels (height $\times$ width). This 2:1 shape is slightly less elongated than most of the crops, but keeps them much closer to their original shape than a square input would. Inception-v3 is the exception: it expects larger inputs, and every image is resized to 299×299 pixels for it, so the aerobic zones are stretched vertically to a square for this architecture. After resizing, every image is normalised with the mean and standard deviation of the ImageNet images, per colour channel, since the pretrained backbones were trained on inputs normalised in this way.

5.3 Augmentation

Data augmentation makes small, random changes to the training images, so that the network sees a slightly different version of every image each time it is used. This increases the variety in the training data without collecting any new images, and makes it harder for the network to memorise individual images. Augmentation is applied to the training set only. The development and test images are never augmented, so that they are evaluated exactly as they are.

The augmentations are applied by PyTorch each time an image is loaded, rather than being generated once and saved to disk. The size of the training set therefore stays the same, but every epoch sees a new, randomly augmented version of each training image. Three augmentations are applied, in the following order:

- **Horizontal flip**: the image is mirrored from left to right with a probability of 50%.
- **Rotation**: the image is rotated by a random angle, chosen uniformly between $-15°$

and +15°. The output image keeps its original size, so the corners that are rotated out of the frame are cut off, and the empty corners that are rotated into the frame are filled with black.

- **Colour jitter**: the brightness, contrast, saturation and hue of the image are changed by a random amount, with the four adjustments applied in a random order. The brightness and contrast are each multiplied by a random factor between 0.6 and 1.4, the saturation by a random factor between 0.7 and 1.3, and the hue is shifted by up to ±0.1 of the full colour wheel (at most 36°).

The flip and rotation are suited to this dataset because the images are taken from directly above. A clarifier seen from above has no fixed orientation, so a mirrored or slightly rotated clarifier is still a realistic example of the same class. When `prep.py` crops a clarifier, it also sets every pixel outside the annotated circle to black, which matches the black corners introduced by the rotation. The aerobic zones do not have this black background, since their annotations are rectangles. For them, a rotation cuts off the ends of the zone and adds small black triangles along its edges. The rotation is limited to ±15°, so most of the surface of the zone remains visible.

The colour jitter targets a different problem. The images come from different facilities and were captured on different dates, so the lighting, sun glare and water colour vary between images. The colour jitter is intended to stop the network from memorising the exact colour or brightness of the water at a particular facility, and to make it rely on other features of the clarifier instead.

The hue shift does raise a concern, since the clarifier classes are partly defined by colour, and a **Dysfunctional** clarifier is identified by its green surface (Chapter 4). A hue shift could wash out this green colour while the image keeps its **Dysfunctional** label. To test whether this harmed the models, all seven clarifier architectures were retrained on all five leave-one-facility-out splits with the hue shift disabled, keeping every other augmentation and hyperparameter the same. Disabling the hue shift did not affect performance, and so it was maintained (Section 6.3.4). The aerobic zones use the same augmentations. Their classes are defined by the leopard print pattern on the surface rather than by its colour (Chapter 4), so the same concern applies less directly, and the hue shift was not tested separately for the aerobic zones.

Figure 5.2 shows these augmentations applied to a single training image. The top row shows each augmentation on its own, while the bottom row shows four randomly chosen outputs of the full augmentation pipeline, as the network would see them during training.

5.4 Class Imbalance

As shown in Tables 4.3 and 4.4 in Section 4.8, the classes of both components are highly imbalanced. For the clarifiers, **Functional** and **Scum** clarifiers make up most of the

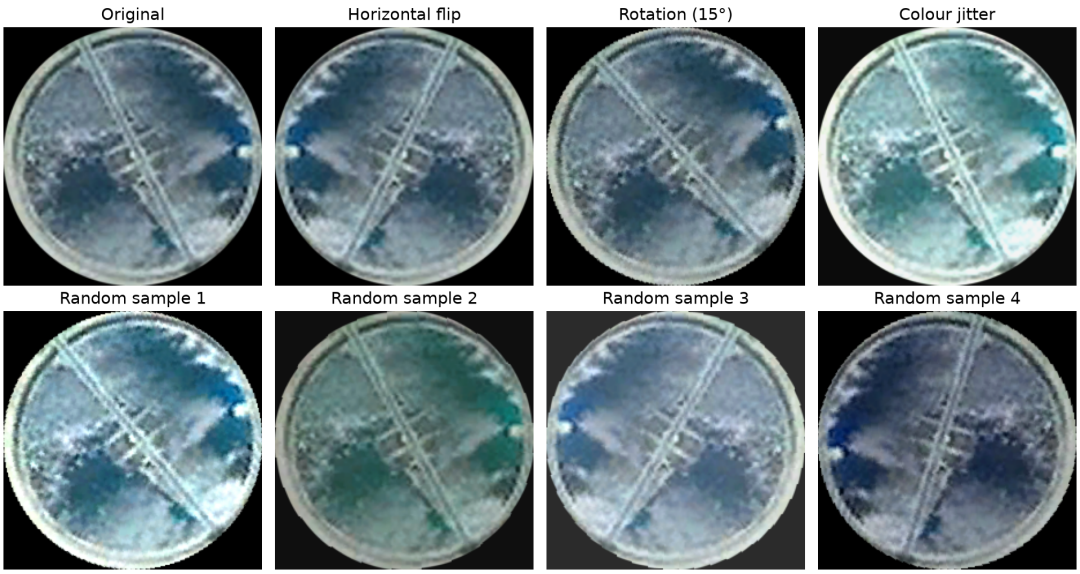


Figure 5.2: Augmentations applied to a single **Scum** clarifier from Cape Flats. Top row: the original image, followed by a horizontal flip, a rotation of 15°, and a random colour jitter, each applied on its own. Bottom row: four random outputs of the full augmentation pipeline (flip, rotation and colour jitter combined).

dataset (2090 of the 2527 images), while **Dysfunctional** and especially **Empty** clarifiers are much less common. For the aerobic zones, **Suboptimal** is the rare class, with only 223 of the 1743 images. If every image contributes equally to the loss, the network can reach a low loss by mostly predicting the common classes, and has little incentive to learn the rare ones.

To counter this, the cross-entropy loss is weighted per class, so that the network is penalised more for misclassifying an image from a rare class than from a common one. The weight of each class is inversely proportional to the number of training images in that class, and the weights are then normalised so that they sum to the number of classes. For K classes, where n_c is the number of training images in class c, the weight of class c is

$$w_c = K \cdot \frac{1/n_c}{\sum_{j=1}^{K} 1/n_j}. \tag{5.1}$$

A class with average representation therefore gets a weight close to one, a rare class gets a weight larger than one, and a common class gets a weight smaller than one. These weights are passed to PyTorch's `CrossEntropyLoss`, which multiplies the loss of each image by the weight of its true class. The training data itself is not resampled, so every training image is still used exactly once per epoch.

The weights are calculated from the training set only, and are recalculated for every split, since the class counts change depending on which facility is held out. Tables 5.2 and 5.3 list the number of training images per class and the resulting weights for every leave-one-facility-out split, and for the stratified split of Experiment 1.

Table 5.2: Number of training images per clarifier class and the resulting class weight w_c, for each held-out test facility and for the stratified split.

	Dysfunctional		**Empty**		**Functional**		**Scum**	
Test facility	Images	w_c	Images	w_c	Images	w_c	Images	w_c
Atlantis	236	1.10	116	2.24	769	0.34	822	0.32
Cape Flats	125	0.52	21	3.09	261	0.25	439	0.15
Fisantekraal	222	1.11	110	2.23	717	0.34	780	0.32
Northern Works	131	1.55	107	1.90	740	0.27	729	0.28
Waterval	230	1.12	118	2.18	733	0.35	722	0.36
None (stratified)	199	1.11	97	2.28	699	0.32	757	0.29

Table 5.3: Number of training images per aerobic zone class and the resulting class weight w_c, for each held-out test facility and for the stratified split.

	Dysfunctional		**Functional**		**Suboptimal**	
Test facility	Images	w_c	Images	w_c	Images	w_c
Atlantis	635	0.49	510	0.61	165	1.89
Cape Flats	202	0.82	493	0.34	90	1.84
Waterval	469	0.34	133	1.19	107	1.48
None (stratified)	567	0.51	492	0.59	153	1.90

For the clarifiers, the Cape Flats split stands out. Cape Flats contains 120 of the 145 **Empty** clarifiers in the dataset (Table 4.4 in Section 4.8), so when it is held out for testing, only 25 **Empty** images are left, 21 of which are in the training set. The weight of the **Empty** class rises to 3.09 as a result, about twenty times the weight of the **Scum** class (0.15). In the same way, Northern Works contains 131 of the 292 **Dysfunctional** clarifiers, so holding it out raises the **Dysfunctional** weight to 1.55, compared with about 1.1 in the Atlantis, Fisantekraal and Waterval splits.

For the aerobic zones, **Suboptimal** has the highest weight in every split, between 1.48 and 1.90. The held-out facility again changes the weights of the other two classes. Waterval contains 543 of the 707 **Functional** aerobic zones, so when it is held out, **Functional** becomes a minority class in the training set, with a weight of 1.19 compared with 0.34 to 0.61 in the other splits. The Waterval test set is therefore dominated by a class that is comparatively scarce in its own training set. Similarly, Cape Flats contains 562 of the 813 **Dysfunctional** aerobic zones, and holding it out raises the **Dysfunctional** weight to 0.82, compared with 0.34 to 0.51 in the other splits.

The same weighted loss is also used to calculate the development loss. Since the development loss decides when training stops early and which epoch's weights are kept (Section 5.6), the class weighting also influences model selection, and not only the gradient updates during training.

5.5 Hyperparameter Tuning

Some hyperparameters were fixed for every architecture: Adam's β_2 (0.999, the default from Adam's original paper), the L_2 weight decay (10^{-4}), a maximum of 100 epochs, an early-stopping patience of 10 epochs and a random seed of 42. The remaining hyperparameters, namely the number of hidden layers H and neurons per layer N of the classifier head, the learning rate, Adam's β_1, the `StepLR` step size and the batch size, were tuned with Optuna [36], a hyperparameter optimisation library. This was done for six of the seven architectures, separately for the clarifiers and the aerobic zones, giving twelve searches. AlexNet was not tuned (Section 5.5.3).

5.5.1 Why Optuna

The six tuned hyperparameters interact, so tuning them one at a time can miss good combinations, and a grid search over all six is too expensive. Optuna searches all six jointly with a Tree-structured Parzen Estimator (TPE) sampler, which samples new candidates from the regions of the search space where earlier trials scored well. A `MedianPruner` also stops a trial early if its development accuracy falls below the median of the earlier trials at the same epoch.

5.5.2 Search Procedure

The first ten trials of every search are sampled at random, and the last ten are guided by TPE. The pruner only becomes active after five trials have completed, and does not prune a trial before its fifth epoch. Every trial uses the same training loop as the final models (Section 5.6) and is scored by its development accuracy at the epoch with the lowest development loss, which is the checkpoint that is kept when a model is trained.

To limit the compute cost, each search was run once, on the leave-one-facility-out fold with Northern Works held out, rather than on every fold. The selected values are used for every fold and for the stratified split (Sections 5.7 and 5.8).

Table 5.4 gives the search range of each hyperparameter and the values selected for each architecture. All 240 trials are listed in Appendix A.

5.5.3 AlexNet

AlexNet is trained from randomly initialised weights rather than fine-tuned (Section 5.2). It was not included in the search, and its values were chosen by hand and are the same for both components (Table 5.4). No hidden layers are added (H=0), since AlexNet's own classifier already contains two hidden layers of 4096 neurons. A first configuration with a learning rate of 10^{-3} and three added hidden layers of 1024 neurons often collapsed to predicting a single class, so the learning rate was lowered to 10^{-4} and the added layers were removed. β_1 is set to Adam's default of 0.9. AlexNet is trained for the full 100 epochs without early stopping, since its development loss can stay on a plateau for many epochs

Table 5.4: Search range of each tuned hyperparameter, and the values used for each architecture and component, with the development accuracy of the selected trial. The learning rate is sampled log-uniformly, and the batch size from $\{8, 16, 24, 32, 48, 64, 96, 128\}$. H=0 means the classifier head is a single linear layer, and N is unused. The AlexNet values were chosen by hand (Section 5.5.3).

Architecture	H	N	**LR**	β_1	**Step**	**Batch**	**Dev. acc.**
Search range	$\{0, \ldots, 3\}$	$\{512, 1024, 2048\}$	$[10^{-6}, 10^{-2}]$	$[0.5, 0.999]$	$[5, 40]$	$\{8, \ldots, 128\}$	–
Clarifiers							
ResNet-18	2	2048	3.88×10^{-4}	0.611	12	96	89.27%
ResNet-50	2	2048	3.77×10^{-4}	0.875	17	96	89.02%
DenseNet-121	1	512	3.44×10^{-4}	0.597	11	48	90.24%
Inception-v3	2	2048	1.38×10^{-3}	0.600	23	96	89.51%
EfficientNet-B0	0	–	4.89×10^{-4}	0.857	5	96	88.54%
ConvNeXt-Tiny	1	2048	5.65×10^{-5}	0.560	21	128	90.73%
AlexNet	0	–	1×10^{-4}	0.9	15	32	–
Aerobic zones							
ResNet-18	3	512	2.93×10^{-4}	0.706	24	48	83.87%
ResNet-50	2	2048	4.06×10^{-4}	0.604	23	96	81.82%
DenseNet-121	2	2048	3.16×10^{-4}	0.742	20	8	84.46%
Inception-v3	2	512	2.85×10^{-4}	0.588	33	8	82.40%
EfficientNet-B0	2	2048	1.38×10^{-3}	0.600	23	96	82.99%
ConvNeXt-Tiny	2	2048	6.78×10^{-5}	0.512	27	8	87.39%
AlexNet	0	–	1×10^{-4}	0.9	15	32	–

before it starts to learn, and early stopping ended training too soon. The checkpoint with the lowest development loss is still the one that is kept.

5.6 Training Procedure

Every architecture is trained with the same procedure; only the specific hyperparameter values are different, as set out in Section 5.5.

Rather than using one fixed learning rate for an entire run, every architecture uses `StepLR` to cut the learning rate by a factor of ten every time a fixed, architecture-specific number of epochs (the step size) has passed (Section 5.5), since large steps are useful early on to make fast progress but become counter-productive once the model is close to a good solution, where they cause it to overshoot instead of converging.

Training is allowed to run for up to 100 epochs, but is stopped early if development loss has not improved for 10 consecutive epochs, since the point at which a given architecture stops improving and starts overfitting is not known ahead of time and differs from one architecture to the next. AlexNet is the only exception, and always runs for the full 100 epochs (Section 5.5.3). Development *loss*, rather than accuracy, is used to decide both when to stop and which epoch's weights to keep: whichever epoch achieved the lowest development loss is the one whose weights are restored and carried forward to testing, rather than simply whatever weights training happened to end on. Figure 5.3 summarises this loop.

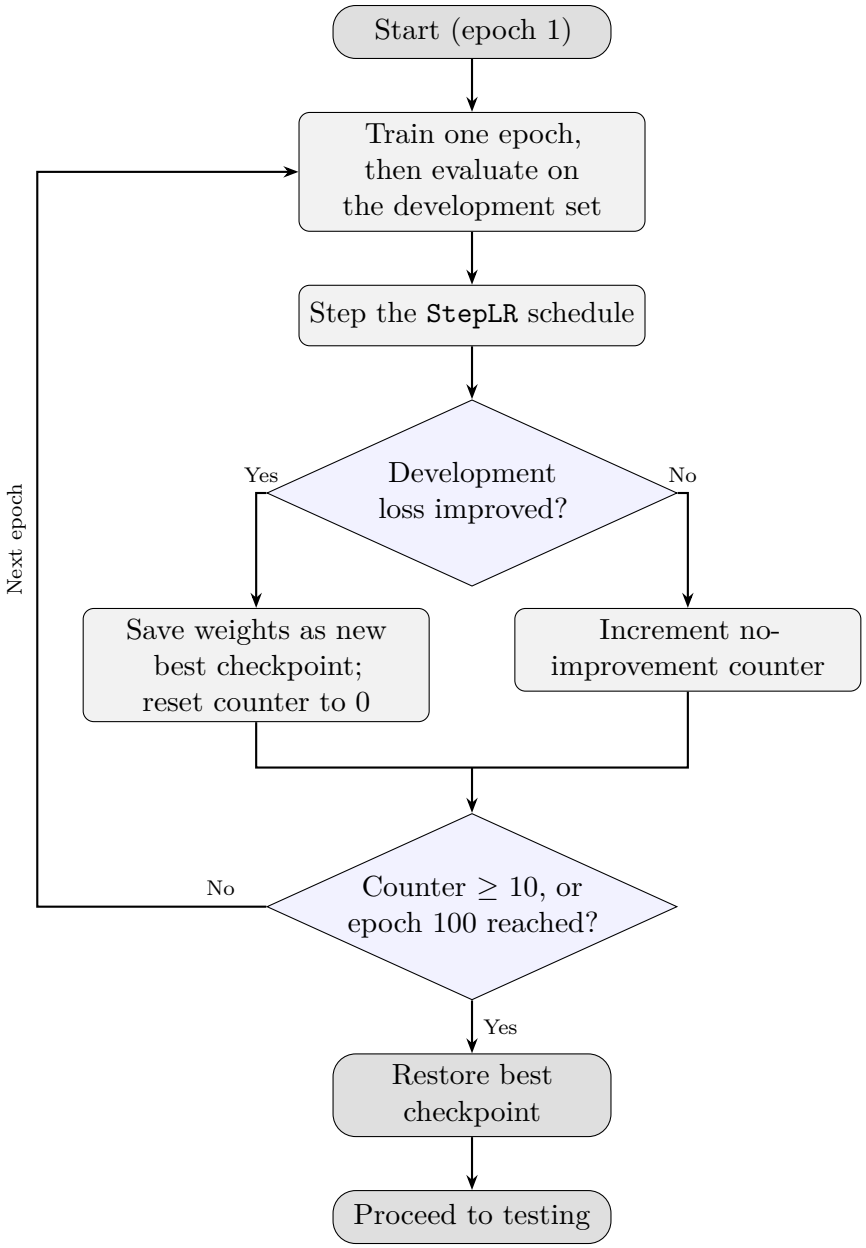


Figure 5.3: The training loop used for every architecture: an epoch of training and development-set evaluation, a scheduler step, a checkpoint decision based on development loss, and a stopping check repeated every epoch until training ends.

Inception-v3 is trained with one additional detail: during training only, it produces a second, auxiliary prediction partway through the network, and the loss from this auxiliary output is added to the main loss at a reduced weight of 0.4. This gives the earlier layers of the network a more direct training signal than would otherwise reach them through such a deep network, and is discarded entirely once training finishes, playing no role in the model's actual predictions at test time.

The same fixed seed used throughout this project (Section 5.5) also covers the random initialisation of each architecture's new classifier head and the order in which the training images are shuffled, in addition to the dataset splitting described in Chapter 4. A repeated run therefore starts from the same weights and splits. It is not guaranteed to reproduce the results exactly, however, since some GPU operations are not deterministic and mixed precision adds small rounding differences, so repeated runs can differ slightly.

5.7 Experiment 1: Mixed-Facility Baseline

The first experiment does not take into account which facility an image comes from. The images of all five facilities are pooled and divided into training, development and test sets using the stratified 70/20/10 split described in Chapter 4. The images are grouped by facility, unit and class, each group is shuffled using the fixed random seed of 42, and the group is then divided in a 70/20/10 ratio. A group with at least three images always receives at least one development image and one test image, even where rounding would otherwise leave it with none. Every facility, unit and class is therefore represented in all three sets, in proportion to the number of images it contributes. Table 5.5 gives the resulting number of images per class.

Table 5.5: Number of images per class in the training, development and test sets of the stratified 70/20/10 split used in Experiment 1.

Component	Class	Train	Development	Test	Total
Clarifier	Dysfunctional	199	56	37	292
	Empty	97	27	21	145
	Functional	699	198	106	1003
	Scum	757	214	116	1087
	Total	1752	495	280	2527
Aerobic zone	Dysfunctional	567	160	86	813
	Functional	492	139	76	707
	Suboptimal	153	42	28	223
	Total	1212	341	190	1743

All seven architectures are trained using this division of the data, for both the clarifiers and the aerobic zones. Both components use the hyperparameter values optimised on the

fold with Northern Works held out (Table 5.4 in Section 5.5).

Experiment 1 is also used to measure the effect of augmentation. Every architecture is trained a second time on the same data but without augmentation (resizing and normalisation is maintained for all train, development and test images). The data, hyperparameters, class weights and random seed are the same in both experiments, to ensurte that augmentation is the only difference. Experiment 1 therefore consists of 28 training runs: seven architectures, for two components, each trained with and without augmentation.

The purpose of this experiment is to measure how well the models perform in the easiest setting, where the test images come from the same facilities as the training images. The test images are taken from the same units and the same range of capture dates as the training images, and therefore share their lighting, water colour and image quality. Some test images might even share the date with development and training images, because the division into the three sets was made on the basis of the cropped images, rather than the satellite images they originate from, and a single satellite image usually contains several clarifiers and aerobic zones.

A high accuracy on this split therefore does not show that a model would work at a facility it has never seen. If the system were used to monitor other WWTW facilities, most of those facilities would not be part of the training data. A model could also learn cues that are specific to one facility or one date, such as the colour of the water at a particular facility, and still perform well on this split, since the same cues are present in the test set. The results of Experiment 1 are therefore treated as a best-case reference rather than as an estimate of how the system would perform in practice, which is the reason for Experiment 2. The difference between the two experiments gives an indication of how much performance is lost when a model is applied to a facility it was not trained on.

5.8 Experiment 2: Leave-One-Facility-Out Evaluation

Experiment 2 uses the leave-one-facility-out split described in Chapter 4 and illustrated in Figure 4.6. One facility is held out as the test set, and the images of the remaining facilities are divided into training and development sets in an 80/20 ratio, grouped by facility, unit and class in the same way as in Experiment 1. This is repeated until every facility has been held out once, and every repetition is referred to as a fold. The images of the test facility are never used to update the weights of the network, to calculate the class weights, to decide when training stops or to select the checkpoint that is kept. The test results of a fold are therefore an estimate of how a model would perform at a facility it has never seen.

Every architecture is trained separately for every fold, using the hyperparameters (Section 5.5), class-weighted loss (Section 5.4), augmentation (Section 5.3) and training loop (Section 5.6) described earlier in this chapter, with the same hyperparameters

determined in experiment 1 used throughout. For the clarifiers, this gives five folds and 35 training runs (one for each architecture). Aerobic zones are only labelled at Atlantis, Cape Flats and Waterval (Chapter 4), so in this case there are only three aerobic zone folds and 21 training runs. Since Fisantekraal and Northern Works contribute no aerobic zone images, every aerobic zone model is trained on the two aerobic facilities that are not held out. Table 5.6 gives the size of every fold, together with the number of test images per class.

Table 5.6: Number of training, development and test images in each leave-one-facility-out fold, and the number of test images per class. The test set is the held-out facility, and the training and development sets are drawn from the remaining facilities.

				Test images per class			
Test facility	**Train**	**Dev.**	**Test**				
Clarifiers				Dysfunctional	Empty	Functional	Scum
Atlantis	1943	474	110	0	3	44	63
Cape Flats	846	205	1476	137	120	679	540
Fisantekraal	1829	446	252	17	9	110	116
Northern Works	1707	415	405	131	13	81	180
Waterval	1803	440	284	7	0	89	188
Aerobic zones				Dysfunctional	Functional	Suboptimal	
Atlantis	1310	320	113	22	72	19	
Cape Flats	785	192	766	562	92	112	
Waterval	709	170	864	229	543	92	

In this experiment the folds differ considerably in size. Cape Flats contains 1476 of the 2527 clarifier images, so when it is held out, the model is trained on only 846 images, less than half of what the other folds are trained on, and tested on the largest test set. Waterval plays a similar role for the aerobic zones, with 864 of the 1743 images. The class composition of the test sets also differs between facilities. The Atlantis test set contains no **Dysfunctional** clarifiers and only three **Empty** clarifiers, and the Waterval test set contains no **Empty** clarifiers and only seven **Dysfunctional** clarifiers, while **Dysfunctional** clarifiers make up almost a third of the Northern Works test set. The recall of a class cannot be calculated at a facility where that class does not occur, and a class with only a handful of test images at a facility leads to an uncertain estimate. For the same reason, the accuracy of one fold cannot be compared directly with that of another: a model that recognises one class well and another poorly will score very differently depending on which class dominates the test set. Macro-averaged metrics, which give every class the same weight, are averaged only over the classes that occur in the test set of a fold. Otherwise, a class without any test images would count as a score of

zero, and would incorrectly lower the macro average of the Atlantis and Waterval clarifier tets sets.

To summarise the performance of an architecture, the results per test fold are averaged, with every fold counting equally regardless of its size. The standard deviation across test folds is reported alongside the mean. Pooling the test images of all folds (micro-averaging) would let Cape Flats determine most of the clarifier result. The standard deviation shows how much the performance of an architecture depends on the facility it is tested on.

One part of this experiment is not fully independent of the test facility. The hyperparameters were tuned once per architecture, with Northern Works held out (Section 5.5), and the development set of that fold contains images from the other four facilities. For the Atlantis, Cape Flats, Fisantekraal and Waterval folds, images from the test facility were therefore part of the development set on which the hyperparameters were chosen, although they were never used to train the model that is tested on them. For the aerobic zones, the tuning development set contains images from all three aerobic facilities, so this applies to every aerobic zone fold. Since only six hyperparameters were chosen in this way, the effect is likely to be small, but only the Northern Works clarifier fold is fully independent of the tuning.

5.9 Experiment 3: Binary (OK / Not-OK) Reformulation

The multi-class models distinguish between several ways in which a component can fail, but for an operator the most important question is whether a component needs attention or not. A **Scum** clarifier classified as **Dysfunctional** is still flagged for inspection, whereas a **Scum** clarifier classified as **Functional** is missed completely. Experiment 3 therefore evaluates the models as binary classifiers, which only decide whether a component is functioning correctly (OK) or not (Not-OK). For the clarifiers, **Functional** means OK, while **Dysfunctional**, **Empty** and **Scum** are Not-OK. For the aerobic zones, **Functional** means OK, while **Suboptimal** and **Dysfunctional** are Not-OK.

No separate binary models are trained. Instead, the binary decision is derived from the multi-class models of Experiments 1 and 2. The Not-OK score of an image is the total probability that the model assigns to the Not-OK classes,

$$s = 1 - P(\text{Functional}), \tag{5.2}$$

and the image is classified as Not-OK when $s \geq t$, for a threshold t. The binary results therefore come from exactly the same models as the multi-class results, so the two can be compared directly. The binary evaluation is carried out on both the stratified split of Experiment 1 and the leave-one-facility-out split of Experiment 2.

With Not-OK treated as the positive class, the recall (or sensitivity) is the fraction of Not-OK components that are flagged as such, while the specificity is the fraction of

OK components that are correctly left unflagged. These two kinds of mistake do not have the same cost. A false alarm only costs an operator the time to look at the image and make his or her own confirmatory assessment, while a missed Not-OK component may go unnoticed until the next satellite image of the facility becomes available. The decision threshold therefore must be chosen so that the recall of the Not-OK class reaches a target of 95%. The recall is evaluated at 500 evenly spaced thresholds between 0 and 1, and the largest threshold that still reaches the target is selected, since this is the threshold that also raises the fewest false alarms. For the leave-one-facility-out models of Experiment 2, one threshold is chosen per architecture and shared by all of its folds: the recall is calculated separately for every held-out facility, and the largest threshold at which the mean recall across facilities reaches 95% is selected.

The ROC curve and its AUC are also calculated for the binary problem, since they describe the trade-off between recall and specificity over all thresholds rather than at a single one. For the leave-one-facility-out models, the binary results are summarised across held-out facilities in the same way as the multi-class results (Section 5.8). The ROC curve of every fold is interpolated onto a common set of false positive rates before the curves are averaged, and the binary confusion matrix at the chosen threshold is normalised per row, so that it shows rates rather than counts, before it is averaged.

Since the threshold is chosen on the test predictions themselves, the recall at the chosen threshold is not an independent result. It is at least 95% by construction on the stratified split, and at least 95% on average across facilities for the leave-one-facility-out split. What this experiment measures is the specificity that has to be accepted to reach this recall and, for the leave-one-facility-out split, how much the recall at one shared threshold varies from one facility to the next. The recall at the shared threshold is therefore also reported for every held-out facility separately. In practice, the threshold would have to be fixed before the system is used at a new facility, so a large spread between facilities would mean that a threshold chosen on known facilities cannot be relied on to reach the target recall at a new one.

5.10 Evaluation Metrics

Every run is evaluated once on its test set, after training has finished and the checkpoint with the lowest development loss has been restored (Section 5.6). A test image is assigned to the class with the highest softmax probability, and the metrics described in Section 2.3 are calculated from these classifications and probabilities.

Of these metrics, recall is the most important in the context of this project. The system is meant to flag components that are not working properly, so that an operator can inspect them. A component with low recall for a failure class, such as a **Dysfunctional** clarifier or a **Suboptimal** aerobic zone, would be missed and could go unnoticed until the next satellite image of the facility, whereas a false positive only costs an operator the time

to look at an image. For this reason, the recall of the failure classes is reported alongside the macro-averaged scores, and the operating point in Experiment 3 is chosen to reach a target recall rather than the highest accuracy.

5.11 Qualitative Analysis: Grad-CAM

Classification performance alone does not show what in an image a model bases its decision on, so a model could rely on something unrelated to the state of the component, such as the water colour at a particular facility or the rim of a clarifier. Grad-CAM (Section 2.4) is used to check this, by highlighting the regions of an image that contributed the most to a prediction.

5.11.1 Method

Grad-CAM was applied to the output of the last convolutional layer of each selected model, using the `pytorch-grad-cam` library [37]. The heatmaps are superimposed on the input images, where red regions contributed the most to the prediction. Every heatmap is normalised separately, so the colours cannot be compared between images.

5.11.2 Models and Example Selection

For each component, the architecture with the highest mean macro-averaged F1 test score in the leave-one-facility-out experiment (Section 5.8) was selected for analysis using one of the held-out test sets. For the clarifiers, Fisantekraal was used, since it contains all four clarifier classes, and for the aerobic zones, Waterval was used, since it has the most aerobic zone images. For each class, one correctly classified image was chosen at random, along with one image from each of the most common mistakes. The heatmaps are calculated for the predicted class, so for a mistake they show what the model took as evidence to arrive at its incorrect decision.

Results

6.1 Overview

6.2 Experiment 1: Mixed-Facility Baseline

6.2.1 Clarifiers

6.2.1.1 Comparison Table

Table 6.1: Clarifier test results of every architecture on the stratified 70/20/10 split (Experiment 1). Macro-averaged scores give every class the same weight. The best value in every column is shown in bold.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.854	0.961	0.855	0.876	0.864
EfficientNet-B0	0.864	0.970	0.859	0.879	0.867
DenseNet-121	0.882	**0.975**	0.891	0.870	0.880
ResNet-50	0.836	0.968	0.812	0.837	0.821
ConvNeXt-Tiny	**0.889**	**0.975**	**0.894**	**0.899**	**0.895**
Inception-v3	0.868	0.968	0.855	0.875	0.864
AlexNet	0.761	0.924	0.781	0.808	0.790

6.2.1.2 Accuracy and AUC

6.2.1.3 Per-Class Precision, Recall and F1

Table 6.2: Per-class precision (P), recall (R) and F1 score of every clarifier architecture on the stratified split (Experiment 1).

	Dysfunctional			Empty			Functional			Scum		
Architecture	P	R	F1	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.895	0.919	0.907	0.826	**0.905**	0.864	0.826	0.896	0.860	0.875	0.784	0.827
EfficientNet-B0	0.868	0.892	0.880	0.826	**0.905**	0.864	0.838	0.925	0.879	0.902	0.793	0.844
DenseNet-121	0.912	0.838	0.873	**0.900**	0.857	0.878	**0.882**	0.915	0.898	0.871	**0.871**	**0.871**
ResNet-50	0.882	0.811	0.845	0.667	0.857	0.750	0.839	0.887	0.862	0.860	0.793	0.825
ConvNeXt-Tiny	**0.944**	0.919	**0.932**	0.864	**0.905**	**0.884**	0.856	**0.953**	**0.902**	**0.913**	0.819	0.864
Inception-v3	0.850	0.919	0.883	0.818	0.857	0.837	0.866	0.915	0.890	0.887	0.810	0.847
AlexNet	0.857	**0.973**	0.911	0.773	0.810	0.791	0.707	0.821	0.760	0.785	0.629	0.699

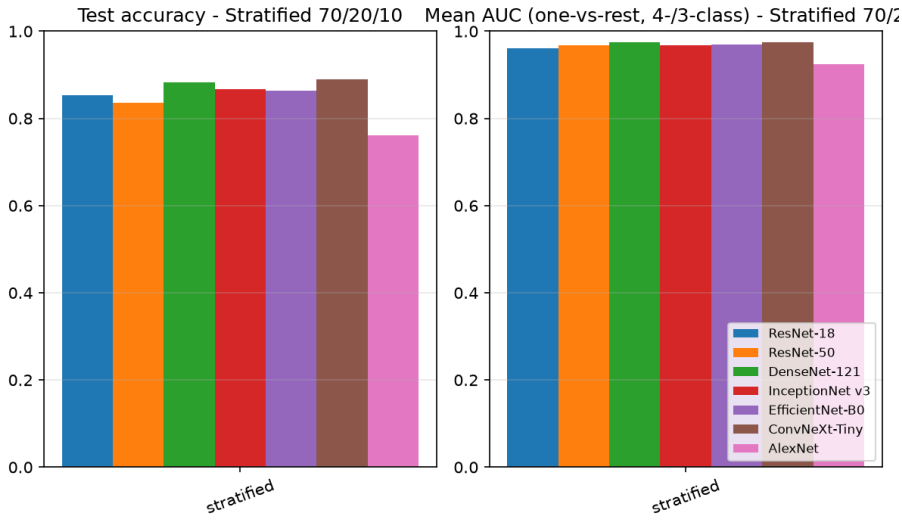


Figure 6.1: Test accuracy and mean one-vs-rest AUC of every clarifier architecture on the stratified split (Experiment 1).

6.2.2 Aerobic Zones

6.2.2.1 Comparison Table

Table 6.3: Aerobic zone test results of every architecture on the stratified 70/20/10 split (Experiment 1).

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.768	0.931	0.736	0.755	0.725
EfficientNet-B0	0.805	0.931	0.758	**0.784**	0.760
DenseNet-121	**0.826**	**0.932**	**0.762**	0.772	**0.765**
ResNet-50	0.811	0.918	0.740	0.749	0.743
ConvNeXt-Tiny	0.795	0.914	0.735	0.746	0.734
Inception-v3	0.784	0.921	0.718	0.730	0.720
AlexNet	0.705	0.884	0.703	0.717	0.675

6.2.2.2 Accuracy and AUC

6.2.2.3 Per-Class Precision, Recall and F1

6.2.3 Effect of Augmentation

Table 6.4: Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture on the stratified split (Experiment 1).

	Dysfunctional			**Functional**			**Suboptimal**		
Architecture	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.933	0.814	0.870	**0.889**	0.737	0.806	0.385	0.714	0.500
EfficientNet-B0	0.973	0.849	**0.907**	0.845	0.789	0.816	0.455	0.714	**0.556**
DenseNet-121	0.924	0.849	0.885	0.861	**0.895**	**0.877**	**0.500**	0.571	0.533
ResNet-50	0.916	**0.884**	0.899	0.863	0.829	0.846	0.441	0.536	0.484
ConvNeXt-Tiny	**0.986**	0.826	0.899	0.810	0.842	0.826	0.410	0.571	0.478
Inception-v3	0.911	0.837	0.873	0.849	0.816	0.832	0.395	0.536	0.455
AlexNet	0.933	0.651	0.767	0.838	0.750	0.792	0.339	**0.750**	0.467

Table 6.5: Test results on the stratified split with (Aug.) and without (None) augmentation during training. Δ is the difference in percentage points, where a positive value means that augmentation helped.

	Accuracy			**Macro F1**			**Mean AUC**	
Architecture	Aug.	None	Δ	Aug.	None	Δ	Aug.	None
Clarifiers								
ResNet-18	0.854	0.850	+0.4	0.864	0.875	-1.1	0.961	0.955
EfficientNet-B0	0.864	0.843	+2.1	0.867	0.856	+1.1	0.970	0.962
DenseNet-121	0.882	0.843	+3.9	0.880	0.818	+6.2	0.975	0.958
ResNet-50	0.836	0.786	+5.0	0.821	0.790	+3.1	0.968	0.944
ConvNeXt-Tiny	0.889	0.886	+0.4	0.895	0.875	+2.0	0.975	0.975
Inception-v3	0.868	0.825	+4.3	0.864	0.828	+3.6	0.968	0.952
AlexNet	0.761	0.750	+1.1	0.790	0.761	+2.9	0.924	0.916
Aerobic zones								
ResNet-18	0.768	0.795	-2.6	0.725	0.736	-1.0	0.931	0.929
EfficientNet-B0	0.805	0.779	+2.6	0.760	0.715	+4.4	0.931	0.919
DenseNet-121	0.826	0.795	+3.2	0.765	0.750	+1.5	0.932	0.909
ResNet-50	0.811	0.789	+2.1	0.743	0.736	+0.7	0.918	0.904
ConvNeXt-Tiny	0.795	0.800	-0.5	0.734	0.753	-1.8	0.914	0.932
Inception-v3	0.784	0.795	-1.1	0.720	0.709	+1.0	0.921	0.894
AlexNet	0.705	0.721	-1.6	0.675	0.658	+1.7	0.884	0.885

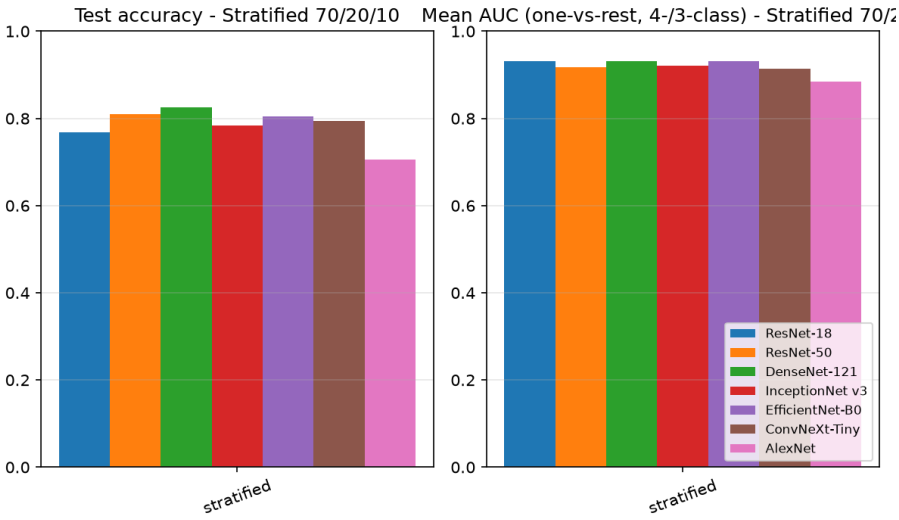


Figure 6.2: Test accuracy and mean one-vs-rest AUC of every aerobic zone architecture on the stratified split (Experiment 1).

6.3 Experiment 2: Leave-One-Facility-Out Evaluation

6.3.1 Clarifiers

6.3.1.1 Comparison Table

Table 6.6: Clarifier test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the five held-out facilities.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.764 ± 0.097	0.914 ± 0.038	0.798 ± 0.096	0.731 ± 0.092	0.739 ± 0.100
EfficientNet-B0	0.804 ± 0.087	0.941 ± 0.030	0.833 ± 0.105	0.747 ± 0.064	0.766 ± 0.079
DenseNet-121	0.802 ± 0.105	0.940 ± 0.041	0.823 ± 0.100	0.742 ± 0.092	0.752 ± 0.092
ResNet-50	0.800 ± 0.086	0.914 ± 0.039	0.815 ± 0.124	0.736 ± 0.059	0.759 ± 0.088
ConvNeXt-Tiny	**0.822** ± 0.080	**0.956** ± 0.018	**0.852** ± 0.088	**0.756** ± 0.079	**0.774** ± 0.077
Inception-v3	0.787 ± 0.106	0.920 ± 0.050	0.795 ± 0.111	0.709 ± 0.073	0.719 ± 0.097
AlexNet	0.687 ± 0.104	0.863 ± 0.041	0.683 ± 0.067	0.693 ± 0.060	0.665 ± 0.065

6.3.1.2 Accuracy and AUC Across Facilities

6.3.1.3 Per-Class Precision, Recall and F1

6.3.1.4 Training Curves and Confusion Matrices

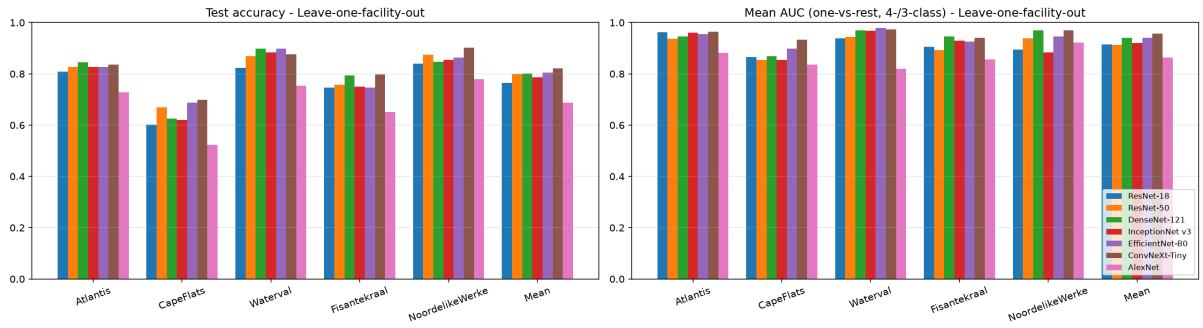


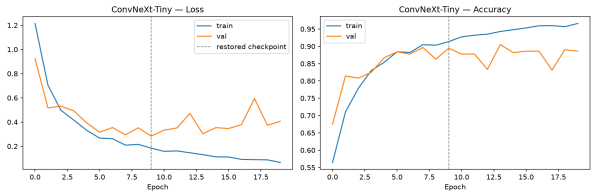
Figure 6.3: Test accuracy and mean one-vs-rest AUC of every clarifier architecture for each held-out facility, and the mean across facilities.

Table 6.7: Clarifier test accuracy and mean AUC of every architecture for each held-out facility.

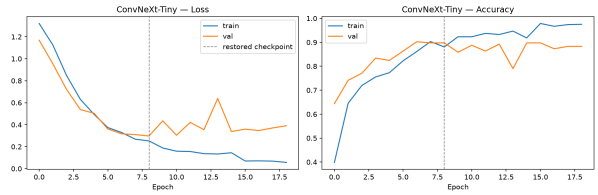
Architecture	Atlantis	Cape Flats	Fisantekraal	Northern Works	Waterval	Mean
Accuracy						
ResNet-18	0.809	0.602	0.746	0.840	0.824	0.764
EfficientNet-B0	0.827	0.687	0.746	0.864	**0.898**	0.804
DenseNet-121	**0.845**	0.626	0.794	0.847	**0.898**	0.802
ResNet-50	0.827	0.669	0.758	0.874	0.870	0.800
ConvNeXt-Tiny	0.836	**0.699**	**0.798**	**0.901**	0.877	**0.822**
Inception-v3	0.827	0.619	0.750	0.854	0.884	0.787
AlexNet	0.727	0.523	0.651	0.780	0.754	0.687
Mean AUC						
ResNet-18	0.963	0.866	0.907	0.895	0.938	0.914
EfficientNet-B0	0.954	0.898	0.926	0.946	**0.978**	0.941
DenseNet-121	0.945	0.869	**0.946**	0.970	0.970	0.940
ResNet-50	0.937	0.854	0.892	0.940	0.945	0.914
ConvNeXt-Tiny	**0.964**	**0.932**	0.941	**0.971**	0.973	**0.956**
Inception-v3	0.961	0.854	0.930	0.884	0.969	0.920
AlexNet	0.881	0.835	0.857	0.923	0.819	0.863

Table 6.8: Per-class precision (P), recall (R) and F1 score of every clarifier architecture in Experiment 2, averaged over the held-out facilities at which the class occurs.

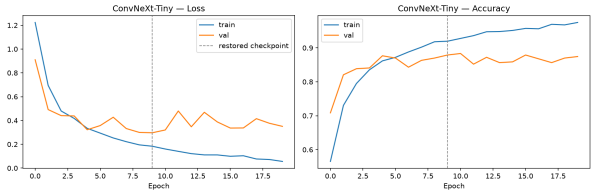
	Dysfunctional			Empty			Functional			Scum		
Architecture	P	R	F1	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.821	0.773	0.793	0.748	0.532	0.604	0.745	0.806	0.740	0.836	0.745	0.765
EfficientNet-B0	0.839	0.767	0.796	0.837	0.586	0.652	0.802	0.790	0.778	0.827	0.812	0.809
DenseNet-121	0.761	0.736	0.741	**0.863**	0.564	0.630	0.816	0.791	0.774	0.835	0.808	0.806
ResNet-50	0.820	0.684	0.733	0.777	0.617	**0.658**	**0.820**	0.781	0.791	0.805	0.808	0.803
ConvNeXt-Tiny	**0.879**	0.749	**0.801**	0.839	0.541	0.613	**0.820**	**0.840**	**0.810**	**0.853**	**0.835**	**0.833**
Inception-v3	0.866	0.685	0.754	0.626	0.495	0.482	0.791	0.812	0.779	0.832	0.779	0.789
AlexNet	0.674	**0.814**	0.732	0.628	**0.633**	0.630	0.664	0.648	0.611	0.746	0.671	0.678



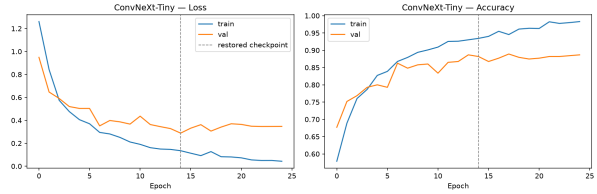
(a) Atlantis held out



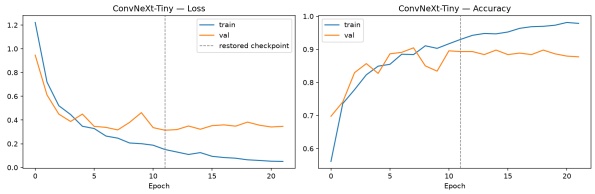
(b) Cape Flats held out



(c) Fisantekraal held out

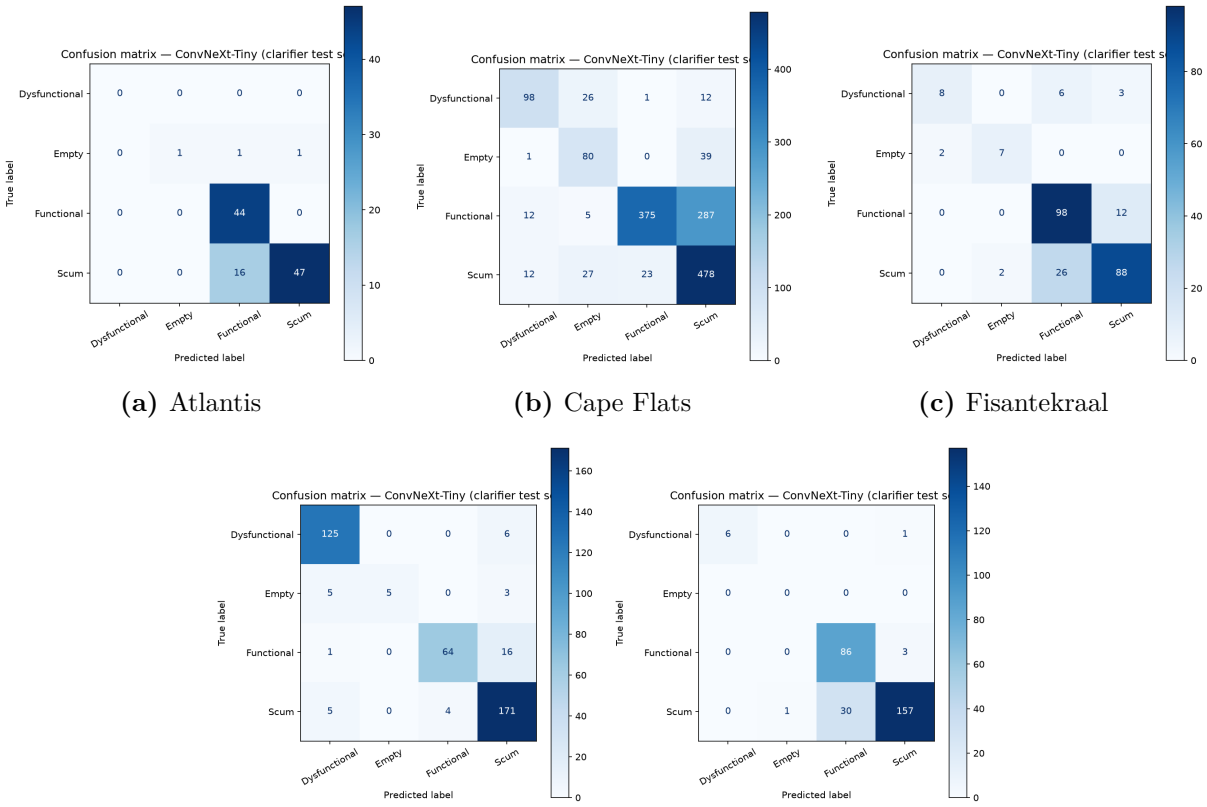


(d) Northern Works held out



(e) Waterval held out

Figure 6.4: Training and development loss and accuracy of the ConvNeXt-Tiny clarifier model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint.



(a) Atlantis **(b)** Cape Flats **(c)** Fisantekraal

(d) Northern Works **(e)** Waterval

Figure 6.5: Confusion matrices of the ConvNeXt-Tiny clarifier model on each held-out facility.

6.3.2 Aerobic Zones

6.3.2.1 Comparison Table

Table 6.9: Aerobic zone test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the three held-out facilities.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.611 ± 0.063	0.722 ± 0.031	0.473 ± 0.033	0.498 ± 0.036	0.444 ± 0.038
EfficientNet-B0	**0.629** ± 0.126	0.733 ± 0.061	0.510 ± 0.043	0.505 ± 0.026	0.469 ± 0.043
DenseNet-121	0.592 ± 0.206	**0.775** ± 0.132	**0.554** ± 0.121	0.529 ± 0.113	**0.498** ± 0.134
ResNet-50	0.467 ± 0.127	0.665 ± 0.050	0.444 ± 0.073	0.446 ± 0.099	0.349 ± 0.102
ConvNeXt-Tiny	0.558 ± 0.147	0.772 ± 0.085	0.520 ± 0.077	**0.537** ± 0.057	0.474 ± 0.097
Inception-v3	0.567 ± 0.161	0.679 ± 0.172	0.495 ± 0.135	0.492 ± 0.194	0.454 ± 0.196
AlexNet	0.370 ± 0.206	0.601 ± 0.037	0.451 ± 0.078	0.389 ± 0.030	0.292 ± 0.135

6.3.2.2 Accuracy and AUC Across Facilities

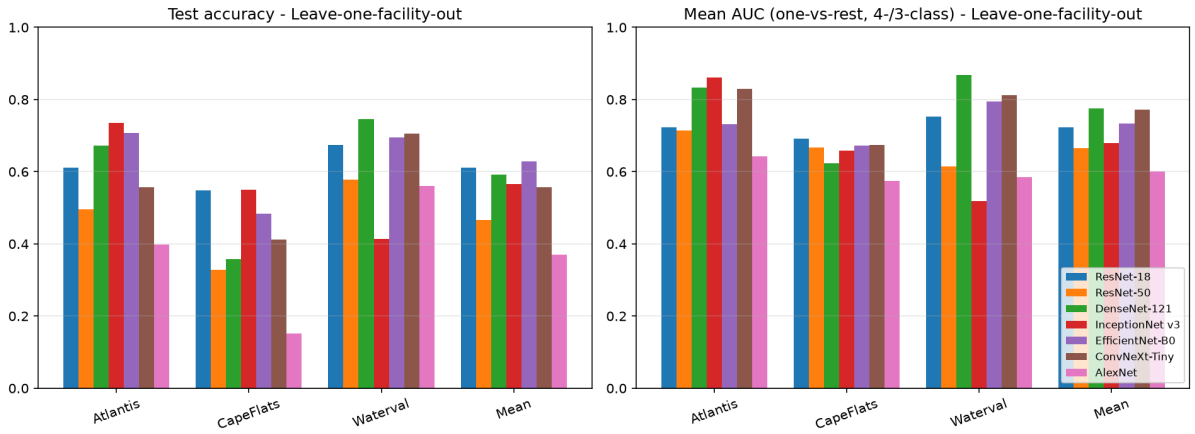


Figure 6.6: Test accuracy and mean one-vs-rest AUC of every aerobic zone architecture for each held-out facility, and the mean across facilities.

6.3.2.3 Per-Class Precision, Recall and F1

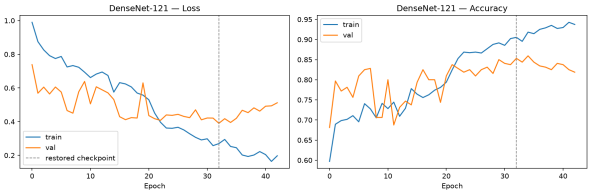
6.3.2.4 Training Curves and Confusion Matrices

Table 6.10: Aerobic zone test accuracy and mean AUC of every architecture for each held-out facility.

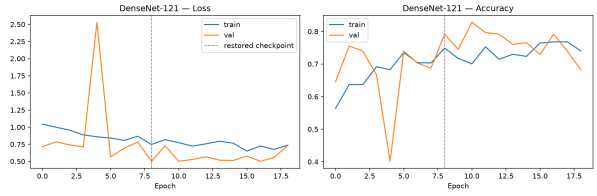
Architecture	Atlantis	Cape Flats	Waterval	Mean
Accuracy				
ResNet-18	0.611	0.548	0.675	0.611
EfficientNet-B0	0.708	0.483	0.696	**0.629**
DenseNet-121	0.673	0.358	**0.747**	0.592
ResNet-50	0.496	0.328	0.578	0.467
ConvNeXt-Tiny	0.558	0.411	0.705	0.558
Inception-v3	**0.735**	**0.551**	0.414	0.567
AlexNet	0.398	0.151	0.561	0.370
Mean AUC				
ResNet-18	0.723	**0.691**	0.753	0.722
EfficientNet-B0	0.731	0.673	0.795	0.733
DenseNet-121	0.833	0.624	**0.869**	**0.775**
ResNet-50	0.714	0.668	0.614	0.665
ConvNeXt-Tiny	0.830	0.674	0.813	0.772
Inception-v3	**0.861**	0.658	0.519	0.679
AlexNet	0.643	0.575	0.585	0.601

Table 6.11: Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture in Experiment 2, averaged over the three held-out facilities.

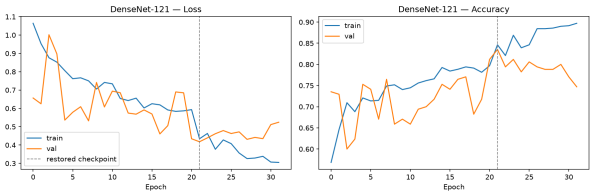
	Dysfunctional			Functional			Suboptimal		
Architecture	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.599	0.578	0.558	0.616	**0.820**	0.654	0.204	0.096	0.121
EfficientNet-B0	0.624	**0.582**	**0.575**	0.655	0.747	**0.691**	0.250	0.185	0.141
DenseNet-121	**0.719**	0.557	0.569	0.648	0.762	0.682	**0.294**	0.268	0.244
ResNet-50	0.591	0.477	0.395	0.602	0.729	0.524	0.140	0.130	0.126
ConvNeXt-Tiny	0.690	0.573	0.551	**0.669**	0.739	0.635	0.201	0.298	0.237
Inception-v3	0.662	0.502	0.497	0.595	0.632	0.600	0.227	**0.341**	**0.263**
AlexNet	0.693	0.163	0.204	0.547	0.715	0.512	0.113	0.288	0.160



(a) Atlantis held out

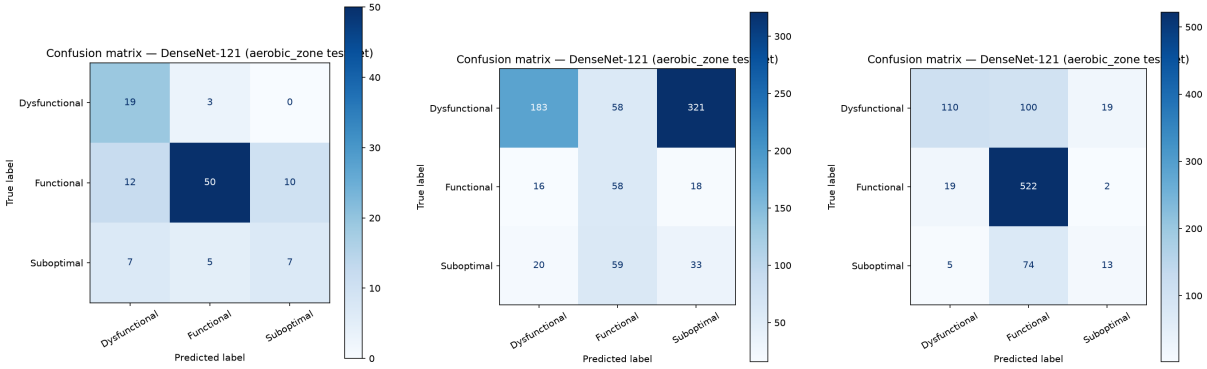


(b) Cape Flats held out



(c) Waterval held out

Figure 6.7: Training and development loss and accuracy of the DenseNet-121 aerobic zone model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint.



(a) Atlantis **(b)** Cape Flats **(c)** Waterval

Figure 6.8: Confusion matrices of the DenseNet-121 aerobic zone model on each held-out facility.

6.3.3 Generalisation: Mixed-Facility Baseline vs Leave-One-Facility-Out

Table 6.12: Test accuracy and macro F1 score on the stratified split (Experiment 1) and the mean across held-out facilities (Experiment 2). Δ is the change in percentage points from Experiment 1 to Experiment 2.

	Accuracy			**Macro F1**		
Architecture	Exp. 1	Exp. 2	Δ	Exp. 1	Exp. 2	Δ
Clarifiers						
ResNet-18	0.854	0.764	-8.9	0.864	0.739	-12.5
EfficientNet-B0	0.864	0.804	-6.0	0.867	0.766	-10.1
DenseNet-121	0.882	0.802	-8.0	0.880	0.752	-12.8
ResNet-50	0.836	0.800	-3.6	0.821	0.759	-6.2
ConvNeXt-Tiny	0.889	0.822	-6.7	0.895	0.774	-12.1
Inception-v3	0.868	0.787	-8.1	0.864	0.719	-14.6
AlexNet	0.761	0.687	-7.4	0.790	0.665	-12.5
Aerobic zones						
ResNet-18	0.768	0.611	-15.7	0.725	0.444	-28.1
EfficientNet-B0	0.805	0.629	-17.6	0.760	0.469	-29.0
DenseNet-121	0.826	0.592	-23.4	0.765	0.498	-26.7
ResNet-50	0.811	0.467	-34.4	0.743	0.349	-39.4
ConvNeXt-Tiny	0.795	0.558	-23.7	0.734	0.474	-26.0
Inception-v3	0.784	0.567	-21.8	0.720	0.454	-26.6
AlexNet	0.705	0.370	-33.5	0.675	0.292	-38.3

6.3.4 Hue Shift

To check whether the hue shift in the colour jitter augmentation (Section 5.3) was washing out the green surface that identifies a **Dysfunctional** clarifier, all seven clarifier architectures were retrained on all five leave-one-facility-out splits with the hue shift disabled, keeping every other augmentation and hyperparameter the same. This did not improve performance: the mean test accuracy decreased for every architecture, by between 0.1 and 3.1 percentage points for the six pretrained architectures, and the number of correctly identified **Dysfunctional** clarifiers did not improve consistently. The hue shift was therefore kept for both components (Section 5.3).

Table 6.13: Mean clarifier test accuracy across the five held-out facilities, and the total number of correctly classified **Dysfunctional** clarifiers (out of 273), with and without the hue shift. Δ is the change in percentage points when the hue shift is removed. These runs were made before the clarifier labels were reviewed.

	Mean accuracy			Correct Dysfunctional	
Architecture	With	Without	Δ	With	Without
ResNet-18	0.782	0.754	-2.8	216	215
EfficientNet-B0	0.806	0.791	-1.5	212	213
DenseNet-121	0.814	0.783	-3.1	216	204
ResNet-50	0.801	0.777	-2.5	214	187
ConvNeXt-Tiny	0.842	0.829	-1.3	226	230
Inception-v3	0.777	0.777	-0.1	200	212
AlexNet	0.496	0.355	-14.0	7	31

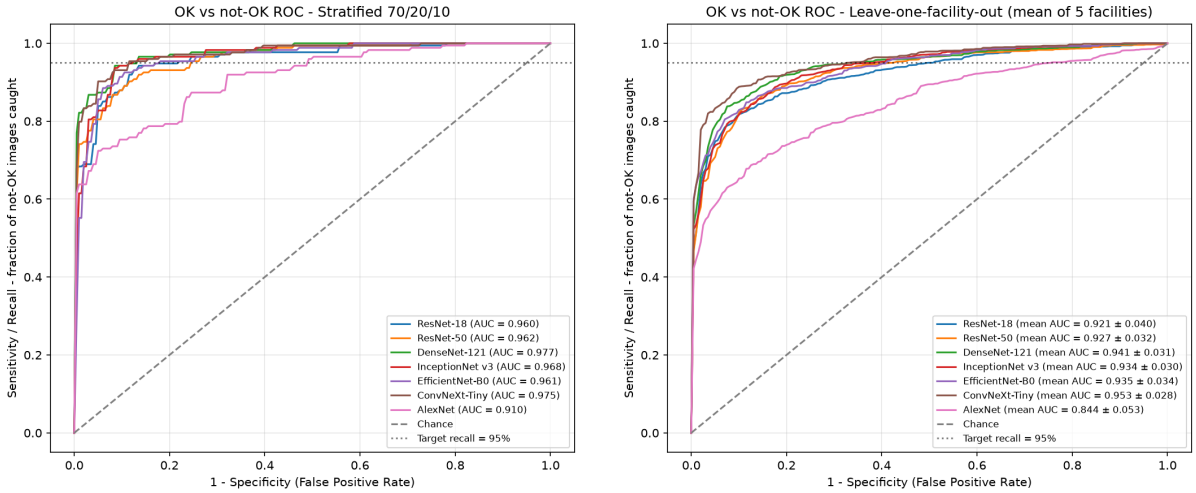
6.4 Experiment 3: Binary (OK / Not-OK) Reformulation

6.4.1 Clarifiers

Table 6.14: Binary OK / Not-OK AUC of every clarifier architecture on the stratified split, and the mean ± standard deviation across held-out facilities.

Architecture	**Stratified**	**Leave-one-facility-out**
ResNet-18	0.960	0.921 ± 0.045
EfficientNet-B0	0.961	0.935 ± 0.038
DenseNet-121	**0.977**	0.941 ± 0.035
ResNet-50	0.962	0.927 ± 0.036
ConvNeXt-Tiny	0.975	**0.953** ± 0.031
Inception-v3	0.968	0.934 ± 0.034
AlexNet	0.910	0.844 ± 0.059

6.4.2 Aerobic Zones



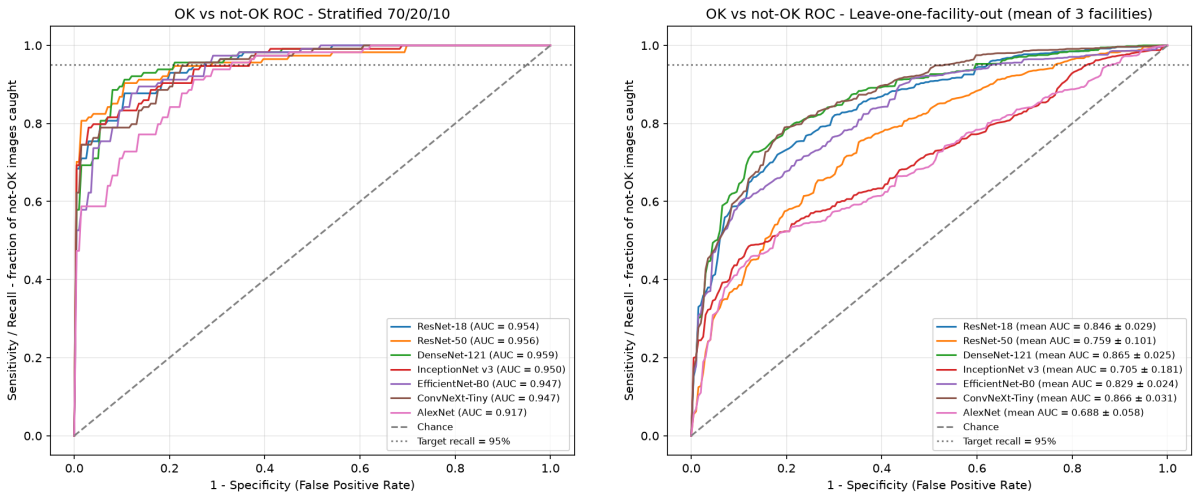
(a) Stratified split

(b) Leave-one-facility-out (mean)

Figure 6.9: Binary OK / Not-OK ROC curves of the clarifier models. The dotted line marks the 95% target recall. The legend in (b) gives the population standard deviation, so it differs slightly from Table 6.14.

Table 6.15: Binary OK / Not-OK AUC of every aerobic zone architecture on the stratified split, and the mean ± standard deviation across held-out facilities.

Architecture	Stratified	Leave-one-facility-out
ResNet-18	0.954	0.846 ± 0.036
EfficientNet-B0	0.947	0.829 ± 0.029
DenseNet-121	**0.959**	0.865 ± 0.030
ResNet-50	0.956	0.759 ± 0.123
ConvNeXt-Tiny	0.947	**0.866** ± 0.038
Inception-v3	0.950	0.705 ± 0.221
AlexNet	0.917	0.688 ± 0.071



(a) Stratified split

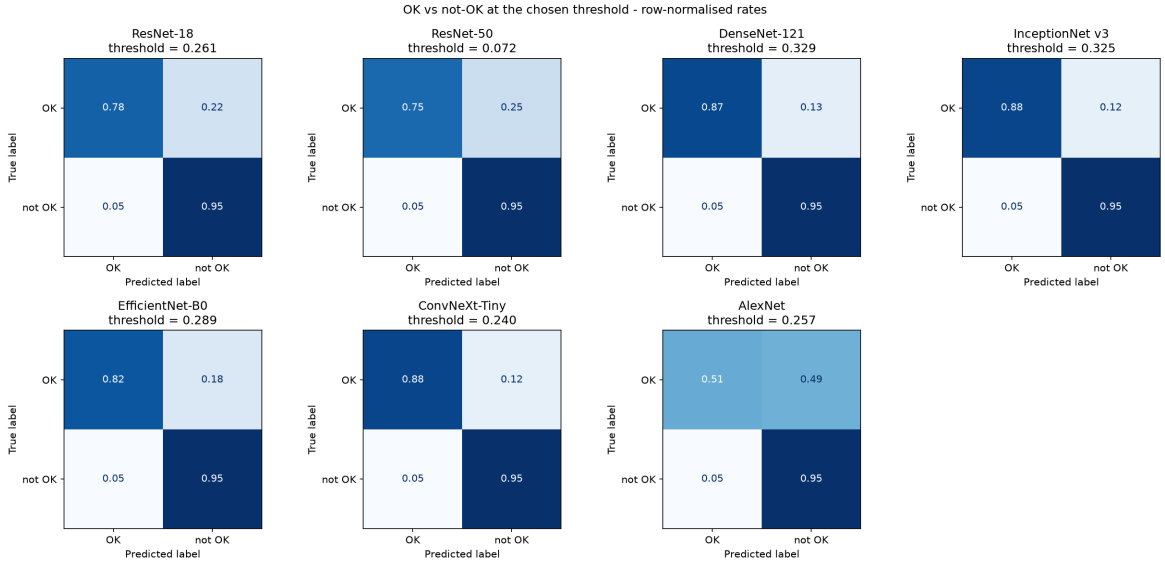
(b) Leave-one-facility-out (mean)

Figure 6.10: Binary OK / Not-OK ROC curves of the aerobic zone models. The dotted line marks the 95% target recall.

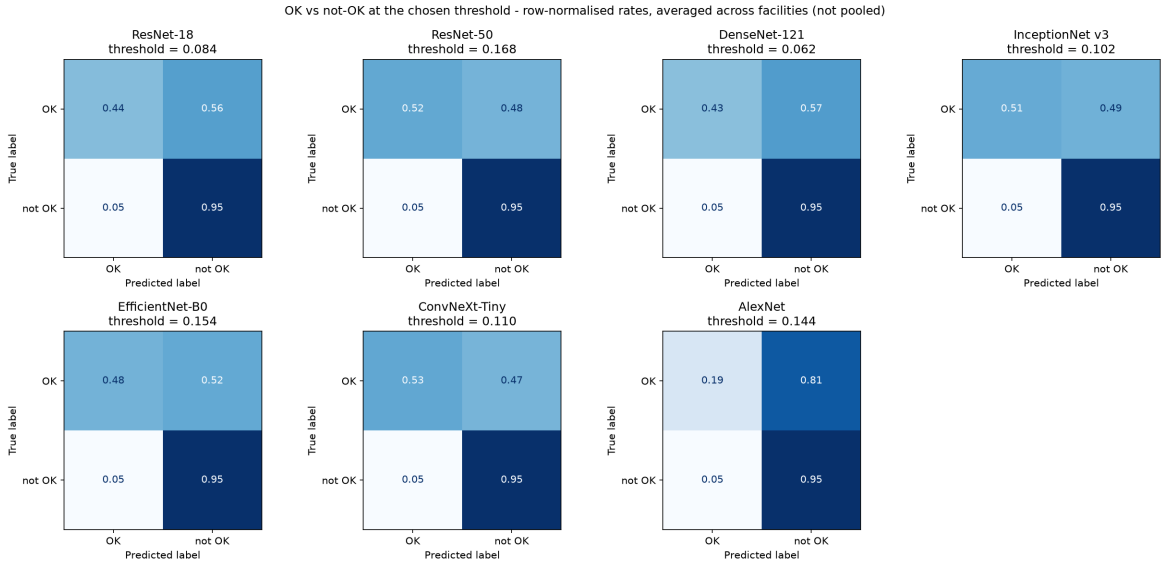
6.4.3 Choosing an Operating Threshold

Table 6.16: Threshold t on the Not-OK score chosen for a 95% recall, with the recall, specificity and precision of the Not-OK class at that threshold. For the leave-one-facility-out split, one threshold is shared by all folds and the metrics are averaged across held-out facilities.

	Stratified				**Leave-one-facility-out**			
Architecture	t	Recall	Spec.	Prec.	t	Recall	Spec.	Prec.
Clarifiers								
ResNet-18	0.261	0.954	0.783	0.878	0.084	0.951	0.441	0.768
EfficientNet-B0	0.289	0.954	0.821	0.897	0.154	0.951	0.480	0.784
DenseNet-121	0.329	0.954	0.868	0.922	0.062	0.950	0.432	0.780
ResNet-50	0.072	0.954	0.755	0.865	0.168	0.953	0.524	0.788
ConvNeXt-Tiny	0.240	0.954	0.877	0.927	0.110	0.950	0.528	0.797
Inception-v3	0.325	0.954	0.877	0.927	0.102	0.952	0.511	0.794
AlexNet	0.257	0.954	0.509	0.761	0.144	0.953	0.190	0.685
Aerobic zones								
ResNet-18	0.571	0.956	0.724	0.838	0.100	0.950	0.269	0.574
EfficientNet-B0	0.343	0.956	0.724	0.838	0.236	0.957	0.325	0.614
DenseNet-121	0.387	0.956	0.776	0.865	0.158	0.952	0.408	0.665
ResNet-50	0.222	0.956	0.711	0.832	0.134	0.952	0.168	0.557
ConvNeXt-Tiny	0.144	0.956	0.763	0.858	0.074	0.951	0.409	0.634
Inception-v3	0.086	0.956	0.632	0.796	0.146	0.950	0.259	0.602
AlexNet	0.385	0.956	0.671	0.813	0.150	0.953	0.137	0.551

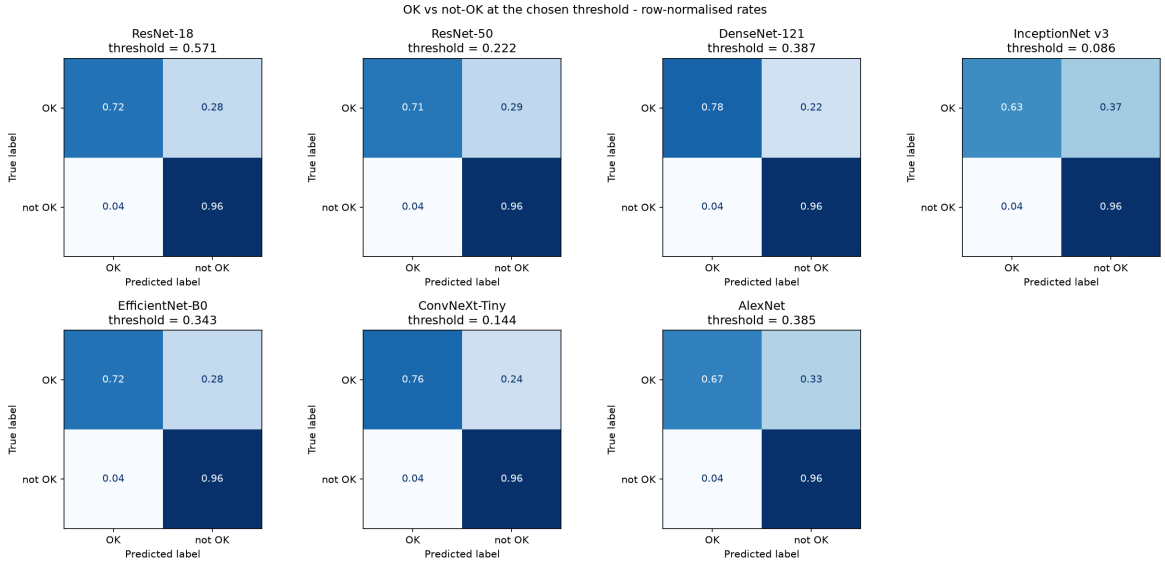


(a) Stratified split

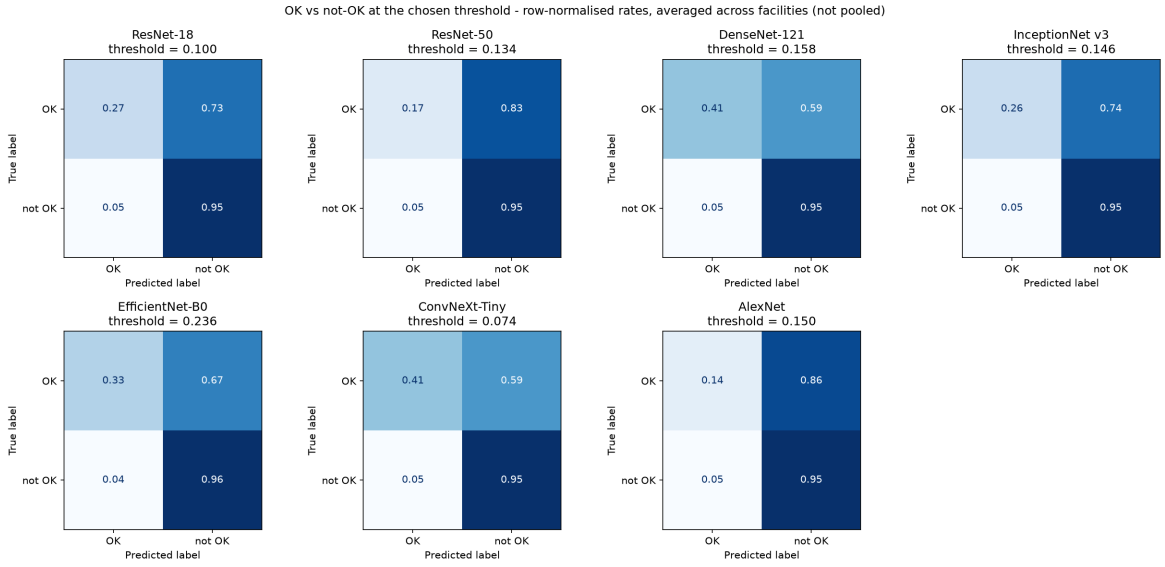


(b) Leave-one-facility-out, averaged across held-out facilities

Figure 6.11: Row-normalised binary confusion matrices of the clarifier models at the chosen threshold.



(a) Stratified split



(b) Leave-one-facility-out, averaged across held-out facilities

Figure 6.12: Row-normalised binary confusion matrices of the aerobic zone models at the chosen threshold.

6.4.4 Threshold Robustness Across Facilities

Table 6.17: Recall and specificity of the Not-OK class for each held-out facility at the shared clarifier threshold of Table 6.16, with the lowest recall and the standard deviation of the recall across facilities.

Architecture	**Atlantis**	**Cape Flats**	**Fisante-kraal**	**Northern Works**	**Waterval**	**Min.**	**Std.**
Recall							
ResNet-18	0.909	0.995	0.901	1.000	0.949	0.901	0.046
EfficientNet-B0	0.833	0.996	0.944	1.000	0.979	0.833	0.069
DenseNet-121	0.894	0.997	0.859	1.000	1.000	0.859	0.068
ResNet-50	0.909	0.974	0.908	0.994	0.979	0.908	0.041
ConvNeXt-Tiny	0.848	1.000	0.965	1.000	0.938	0.848	0.063
Inception-v3	0.894	0.992	0.901	0.985	0.990	0.894	0.050
AlexNet	0.894	1.000	0.880	1.000	0.990	0.880	0.060
Specificity							
ResNet-18	0.841	0.080	0.491	0.086	0.708		
EfficientNet-B0	0.955	0.150	0.464	0.247	0.584		
DenseNet-121	0.818	0.108	0.855	0.086	0.292		
ResNet-50	0.705	0.436	0.609	0.210	0.663		
ConvNeXt-Tiny	1.000	0.128	0.391	0.444	0.674		
Inception-v3	0.864	0.168	0.773	0.358	0.393		
AlexNet	0.523	0.006	0.409	0.000	0.011		

Table 6.18: Recall and specificity of the Not-OK class for each held-out facility at the shared aerobic zone threshold of Table 6.16, with the lowest recall and the standard deviation of the recall across facilities.

Architecture	Atlantis	Cape Flats	Waterval	Min.	Std.
Recall					
ResNet-18	1.000	0.850	1.000	0.850	0.087
EfficientNet-B0	0.951	0.926	0.994	0.926	0.034
DenseNet-121	0.951	0.964	0.941	0.941	0.012
ResNet-50	1.000	0.856	1.000	0.856	0.083
ConvNeXt-Tiny	1.000	0.933	0.919	0.919	0.043
Inception-v3	0.951	0.967	0.931	0.931	0.018
AlexNet	1.000	0.858	1.000	0.858	0.082
Specificity					
ResNet-18	0.083	0.630	0.094		
EfficientNet-B0	0.361	0.239	0.376		
DenseNet-121	0.403	0.152	0.670		
ResNet-50	0.069	0.413	0.020		
ConvNeXt-Tiny	0.250	0.424	0.554		
Inception-v3	0.500	0.109	0.168		
AlexNet	0.028	0.380	0.004		

6.5 Qualitative Analysis: Grad-CAM

The Grad-CAM method and example selection described in Section 5.11 were applied to the architecture with the highest mean macro-averaged F1 score in the leave-one-facility-out experiment for each component: ConvNeXt-Tiny for the clarifiers, tested on Fisantekraal (79.8% accuracy), and DenseNet-121 for the aerobic zones, tested on Waterval (74.7% accuracy). The last convolutional layer of these networks is only a 7×7 and 3×7 grid respectively, so the heatmaps are coarse.

6.5.1 Clarifiers

Figure 6.13 shows the examples for the clarifiers. For the correctly classified **Dysfunctional**, **Empty** and **Scum** clarifiers, the most important region lies on the surface inside the tank. The **Dysfunctional** heatmap is on the green surface. The **Empty** heatmap is on the edge of the dark shadow at the top of the tank and on the bright floor below it, which matches the labelling rule that shadows are visible inside an empty tank. The **Scum** heatmap is on the lower part of the surface, next to the bridge (the arm that spans the tank). The **Functional** example is the exception: its strongest region is where the bridge meets the rim at the top of the tank, with only weaker regions on the black surface itself.

The ConvNeXt-Tiny heatmaps tend to highlight one or two small regions rather than a larger part of the surface. This is likely related to the global max pooling used in ConvNeXt-Tiny (Section 5.2): only the strongest activation of each feature map reaches the classifier head, so the model is trained to respond to the single strongest occurrence of

a feature, rather than to how much of the surface it covers.

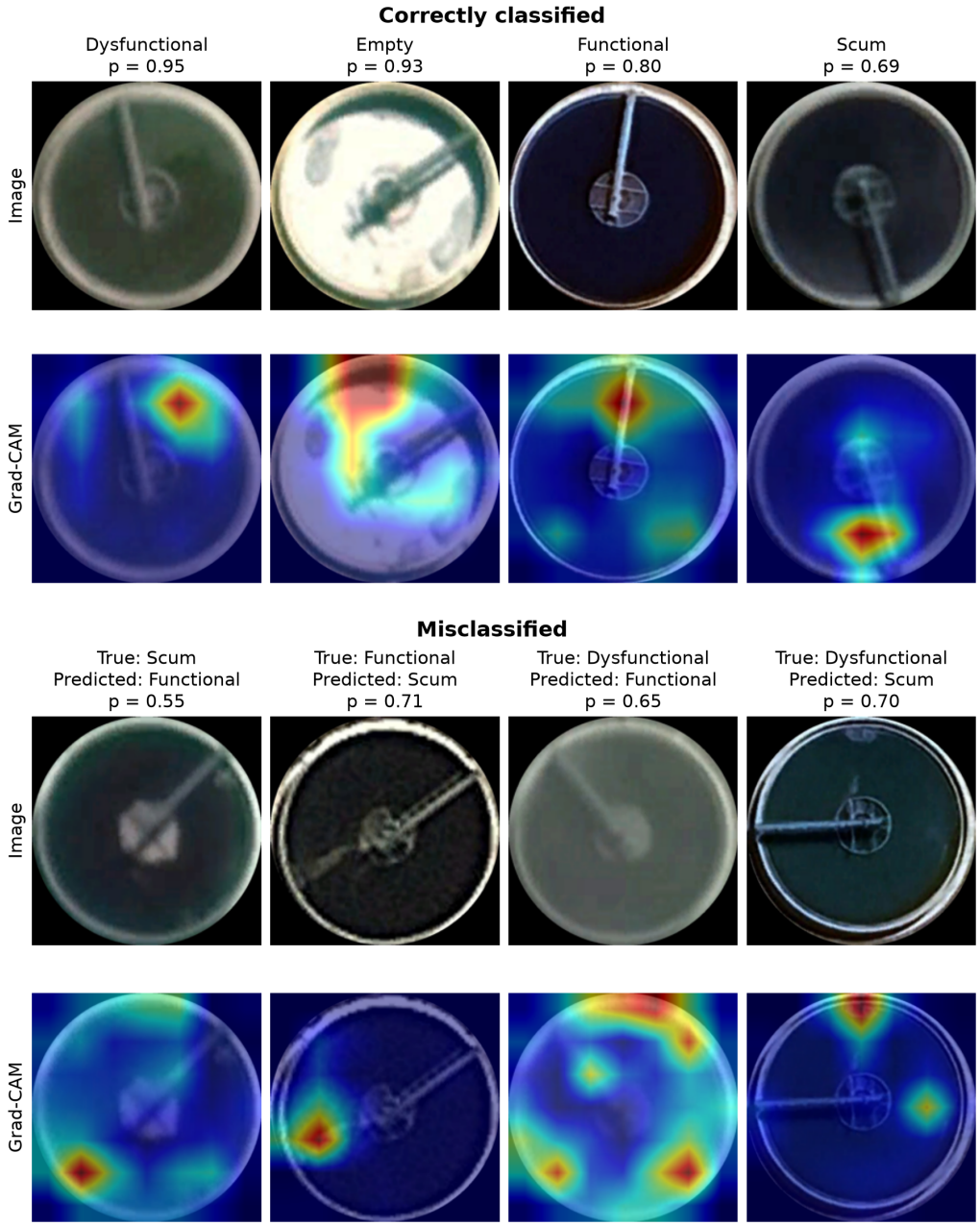


Figure 6.13: Grad-CAM heatmaps of the ConvNeXt-Tiny clarifier model, for test images from Fisantekraal, which was held out during training. Top: one correctly classified clarifier per class. Bottom: one clarifier from each of the four most common mistakes. Every heatmap is calculated for the predicted class, and p is the softmax probability of that class. Red regions contributed the most to the prediction.

The most common mistake at Fisantekraal is a **Scum** clarifier classified as **Functional**, which happened to 26 of the 116 **Scum** clarifiers. In the example, the light patch around the centre of the tank receives almost no weight, and the prediction is based on the black surface near the edge of the tank. That region on its own does look like a **Functional** clarifier, but a clarifier only counts as **Functional** if the whole surface is black. With

$p = 0.55$, the model was also unsure of this prediction. The opposite mistake, a **Functional** clarifier classified as **Scum**, is shown in the second column. This image is grainy and has a high contrast, so the black surface does not look smooth, and the heatmap lies on the surface at the left edge of the tank. Noise in the image itself could therefore be mistaken for a surface that is not uniformly black.

Only 17 of the Fisantekraal clarifiers are **Dysfunctional**, but 9 of them were misclassified, and the last two columns show a different problem. In both, most of the weight lies on or next to the concrete rim of the tank rather than on the surface. For the clarifier classified as **Functional**, part of the heatmap even falls on the black background outside the tank. Neither of these regions says anything about the state of the clarifier. The surface of this clarifier is also a pale grey-green rather than the darker green of the correctly classified **Dysfunctional** example, so there was little evidence on the surface itself. For the clarifier classified as **Scum**, the strongest region is on the rim at the top of the tank, with a weaker region on the surface to the right of the centre.

6.5.2 Aerobic Zones

Figure 6.14 shows the examples for the aerobic zones. The correctly classified **Functional** zone is a clear case. The model is certain of its prediction ($p = 1.00$), and the heatmap covers the centre of the zone, where the leopard print pattern is visible. This is the feature an operator would look for, which suggests that the model has learned to recognise it. The correctly classified **Dysfunctional** and **Suboptimal** zones are much less certain, with $p = 0.49$ and $p = 0.43$, which is only a little above the 0.33 of a random guess between three classes. For the **Dysfunctional** zone, the heatmap covers the smooth surface at the left end of the zone. For the **Suboptimal** zone, it covers a band in the right half of the zone, where the surface is slightly darker and uneven.

Both of the most common mistakes at Waterval are zones classified as **Functional**: 100 of the 229 **Dysfunctional** zones and 74 of the 92 **Suboptimal** zones. The **Suboptimal** example in the second column shows part of the reason. The left and centre of the zone show a clear leopard print pattern, while the right third is smooth. The heatmap lies on the patterned centre, and the smooth part receives almost no weight. According to the labelling rules in Chapter 4, a zone is **Suboptimal** once a third or more of its surface shows no leopard print, so the class depends on how much of the surface is patterned. The model found the pattern where it exists, but did not take into account how much of the surface it covers. This zone is also close to the boundary between the two classes, and since only the clarifier labels were reviewed (Chapter 4), some of these mistakes may partly be caused by the aerobic zone labels themselves.

The **Dysfunctional** zone classified as **Functional**, in the first column, has a smooth surface with no visible pattern, and its heatmap is spread over a large part of the zone rather than on one specific feature. With $p = 0.50$, the model was unsure, and there is

no clear feature in the image that it could have mistaken for the leopard print. For the **Dysfunctional** zone classified as **Suboptimal**, the heatmap is on a darker patch at the right end of the zone, which does not look like the aeration pattern either.

6.5.3 Summary of Observations

When the models were confident and correct, they mostly looked at the features the labels are based on. The mistakes show two problems. For **Scum** and **Suboptimal**, which depend on how much of the surface shows a feature, the models based their prediction on the strongest region and gave little weight to the rest. When the surface gave little evidence either way, the heatmaps spread out or moved to the rim of the tank, and the confidence was low. These are only a few examples, but they are consistent with the most common mistakes at both facilities.

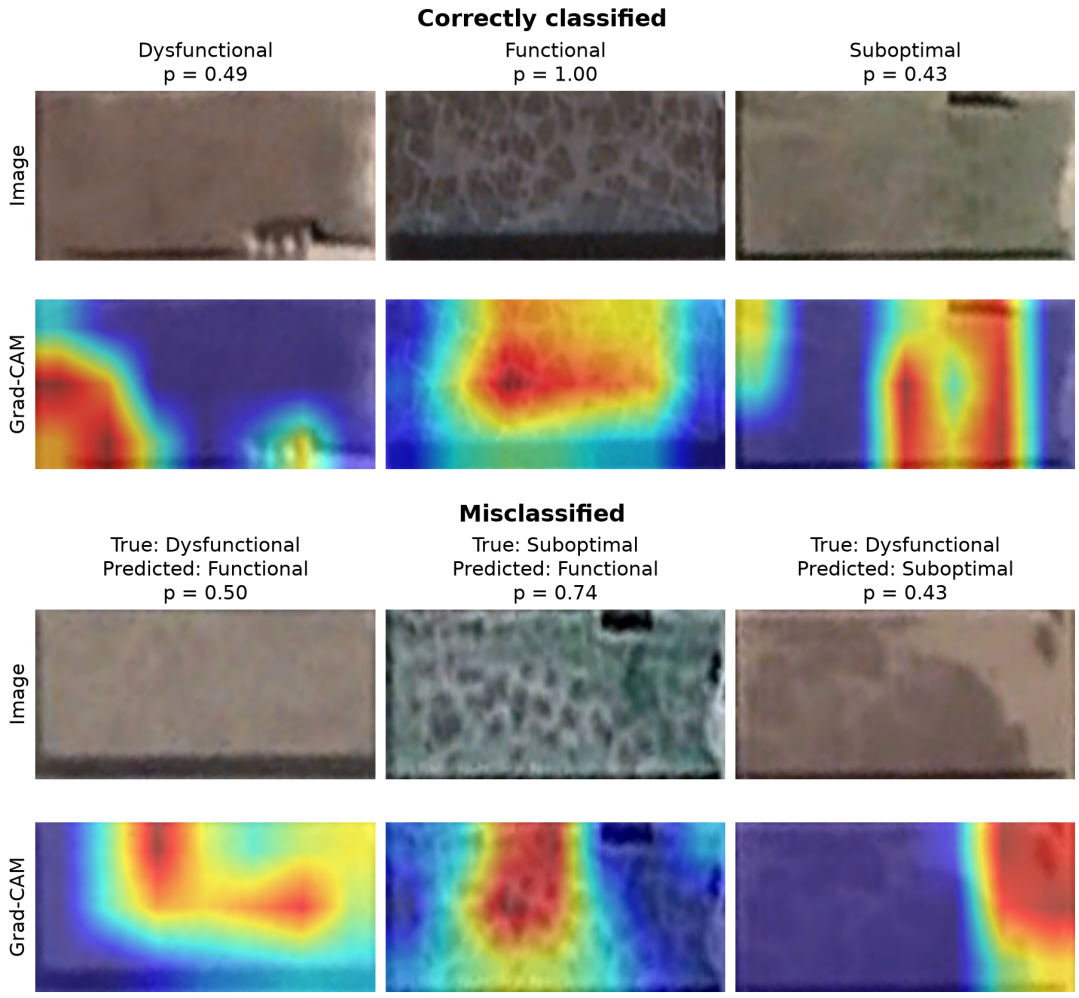


Figure 6.14: Grad-CAM heatmaps of the DenseNet-121 aerobic zone model, for test images from Waterval, which was held out during training. Top: one correctly classified aerobic zone per class. Bottom: one aerobic zone from each of the three most common mistakes. Every heatmap is calculated for the predicted class, and p is the softmax probability of that class. Red regions contributed the most to the prediction.

6.6 Summary of Findings

Chapter 7

Summary and Conclusions

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.

[5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[7] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 976–11 986.

[8] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[9] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in

Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618–626.

[12] Department of Water and Sanitation, "Green drop national report 2022," Department of Water and Sanitation, Pretoria, South Africa, Report, 2022, https://ws.dws.gov.za/IRIS/latestresults.aspx.

[13] J. C. Costa, D. P. Mesquita, A. L. Amaral, M. M. Alves, and E. C. Ferreira, "Quantitative image analysis for the characterization of microbial aggregates in biological wastewater treatment: a review," *Environmental Science and Pollution Research*, vol. 20, pp. 5887–5912, 2013.

[14] S. Borzooei, L. Scabini, G. Miranda, S. Daneshgar, L. Deblieck, O. Bruno, P. De Langhe, B. De Baets, I. Nopens, and E. Torfs, "Evaluation of activated sludge settling characteristics from microscopy images with deep convolutional neural networks and transfer learning," *Journal of Water Process Engineering*, vol. 64, p. 105692, 2024.

[15] H. Satoh, Y. Kashimoto, N. Takahashi, and T. Tsujimura, "Deep learning-based morphology classification of activated sludge flocs in wastewater treatment plants," *Environmental Science: Water Research & Technology*, vol. 7, no. 2, pp. 298–305, 2021.

[16] J. Carballo Mato, S. González Vázquez, J. Fernández Águila, Á. Delgado Rodríguez, X. Lin, L. Garabato Gándara, J. Sobreira Seoane, and J. Silva Castro, "Foam segmentation in wastewater treatment plants," *Water*, vol. 16, no. 3, p. 390, 2024.

[17] M. H. Gholizadeh, A. M. Melesse, and L. Reddi, "A comprehensive review on water quality parameters estimation using remote sensing techniques," *Sensors*, vol. 16, no. 8, p. 1298, 2016.

[18] D. Lopez-Betancur, I. Moreno, C. Guerrero-Mendez, T. Saucedo-Anaya, E. González, C. Bautista-Capetillo, and J. González-Trinidad, "Convolutional neural network for measurement of suspended solids and turbidity," *Applied Sciences*, vol. 12, no. 12, p. 6079, 2022.

[19] M. Lesiv, L. See, J. C. Laso Bayas, T. Sturn, D. Schepaschenko, M. Karner, I. Moorthy, I. McCallum, and S. Fritz, "Characterizing the spatial and temporal availability of very high resolution satellite imagery in Google Earth and Microsoft Bing Maps as a source of reference data," *Land*, vol. 7, no. 4, p. 118, 2018.

[20] Q. Hu, W. Wu, T. Xia, Q. Yu, P. Yang, Z. Li, and Q. Song, "Exploring the use of Google Earth imagery and object-based methods in land use/cover mapping," *Remote Sensing*, vol. 5, no. 11, pp. 6026–6042, 2013.

[21] eoPortal, "QuickBird-2," European Space Agency eoPortal satellite missions directory, 2025, https://www.eoportal.org/satellite-missions/quickbird-2. Accessed 6 October 2026.

[22] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 145, pp. 197–209, 2018.

[23] H. Li, J. Zech, D. Hong, P. Ghamisi, M. Schultz, and A. Zipf, "Leveraging OpenStreetMap and multimodal remote sensing data with joint deep learning for wastewater treatment plants detection," *International Journal of Applied Earth Observation and Geoinformation*, vol. 110, p. 102804, 2022.

[24] Y. Shuai, J. Xie, K. Lu, and Z. Chen, "Multi-attention network for sewage treatment plant detection," *Sustainability*, vol. 15, no. 7, p. 5880, 2023.

[25] S. Randhawa, G. Randhawa, O. Sivak, J. Zech, M. Martin, A. Zipf, and Y. Li, "Multiscale multifeature vision learning for scalable and efficient wastewater treatment plant detection using hi-res satellite imagery and OSM," in *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Advances in Urban-AI (UrbanAI '23)*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 10–21.

[26] C. Handan-Nader and D. E. Ho, "Deep learning to map concentrated animal feeding operations," *Nature Sustainability*, vol. 2, no. 4, pp. 298–306, 2019.

[27] K. Nogueira, O. A. B. Penatti, and J. A. dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognition*, vol. 61, pp. 539–556, 2017.

[28] O. A. B. Penatti, K. Nogueira, and J. A. dos Santos, "Do deep features generalize from everyday objects to remote sensing and aerial scenes domains?" in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2015, pp. 44–51.

[29] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness," in *International Conference on Learning Representations (ICLR)*, 2019, https://arxiv.org/abs/1811.12231.

[30] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018.

[31] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[32] T. Kattenborn, F. Schiefer, J. Frey, H. Feilhauer, M. D. Mahecha, and C. F. Dormann, "Spatially autocorrelated training and validation samples inflate performance assessment of convolutional neural networks," *ISPRS Open Journal of Photogrammetry and Remote Sensing*, vol. 5, p. 100018, 2022.

[33] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[34] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking Clever Hans predictors and assessing what machines really learn," *Nature Communications*, vol. 10, p. 1096, 2019.

[35] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[36] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

[37] J. Gildenblat and contributors, "PyTorch library for CAM methods," 2021, https://github.com/jacobgil/pytorch-grad-cam.

[38] Stellenbosch University, "Formatting your thesis," 2024, http://bit.ly/SUThesisFormat.

[39] D. Els, "Stellenbosch university thesis bundle," 2024, https://ctan.org/pkg/stellenbosch-2.

[40] S. Kretzmann, "'a tsunami of human waste': Half of SA's sewage treatment works are failing, says report," Daily Maverick, 21 June 2022. First published by GroundUp, 2022, https://www.dailymaverick.co.za/article/2022-06-21-a-tsunami-of-human-waste-half-of-sas-sewage-treatment-works-are-failing-says-study/. Accessed 30 September 2026.

[41] A. Westbrook and N. S. de Freitas Alves, "Havana's wastewater treatment plants: Changes over time and estimate of replacement cost," Association for the Study of the Cuban Economy (ASCE). First prize, 2016 ASCE Student Competition for

undergraduate students, 2016, https://ascecubadatabase.org/asce_proceedings/havanas-wastewater-treatment-plants-changes-over-time-and-estimate-of-replacement-cost/. Accessed 30 September 2026.

Hyperparameter Search Trials

Section 5.5 describes the Optuna hyperparameter search used for six of the project's architectures, which was run separately for the clarifiers and for the aerobic zones. This appendix lists every trial of these twelve searches, so the results in that section can be checked in full without cluttering the main text with 240 rows of trial data. For each search, the winning trial (the one whose values were carried forward to training) is shaded.

A.1 Clarifiers

A.1.1 ResNet-18

Table A.1: All 20 clarifier Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	**LR**	β_1	**Step**	**Batch**	**Outcome**	**Dev. acc.**
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	79.51%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	85.85%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	74.63%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	84.39%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	74.15%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	84.63%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	85.12%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	84.88%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	87.56%
10	3	512	1.53×10^{-5}	0.617	33	96	Pruned	–
11	2	2048	3.88×10^{-4}	0.611	12	96	Completed	89.27%
12	3	512	4.16×10^{-4}	0.897	15	96	Completed	88.29%
13	3	512	1.04×10^{-3}	0.654	15	96	Pruned	–
14	2	2048	4.83×10^{-3}	0.679	11	96	Pruned	–
15	1	512	5.70×10^{-4}	0.963	16	96	Completed	88.05%
16	2	2048	1.35×10^{-4}	0.900	21	96	Completed	85.85%
17	2	2048	2.42×10^{-4}	0.663	6	96	Completed	87.56%
18	3	2048	2.00×10^{-4}	0.771	14	16	Pruned	–
19	3	512	1.02×10^{-3}	0.873	24	96	Pruned	–

A.1.2 EfficientNet-B0

Table A.2: All 20 clarifier Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	81.71%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	86.34%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	86.10%
3	1	2048	1.22×10^{-5}	0.914	17	64	Pruned	–
4	1	512	2.92×10^{-3}	0.800	30	16	Completed	81.71%
5	2	2048	1.38×10^{-3}	0.600	23	96	Completed	86.34%
6	2	1024	4.34×10^{-3}	0.629	28	48	Pruned	–
7	1	2048	1.22×10^{-5}	0.914	17	64	Pruned	–
8	1	512	2.91×10^{-6}	0.931	27	96	Pruned	–
9	2	512	5.13×10^{-5}	0.513	8	48	Completed	83.66%
10	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
11	0	2048	3.63×10^{-3}	0.715	6	96	Completed	88.29%
12	0	2048	4.39×10^{-3}	0.743	5	96	Pruned	–
13	0	2048	4.87×10^{-4}	0.682	15	16	Completed	88.29%
14	0	2048	1.40×10^{-4}	0.626	9	16	Pruned	–
15	0	2048	1.23×10^{-4}	0.704	10	24	Completed	87.32%
16	0	2048	4.89×10^{-4}	0.857	5	96	Completed	88.54%
17	0	2048	6.68×10^{-3}	0.975	7	96	Pruned	–
18	0	2048	7.84×10^{-4}	0.901	8	96	Pruned	–
19	0	2048	1.23×10^{-4}	0.841	7	64	Pruned	–

A.1.3 DenseNet-121

Table A.3: All 20 clarifier Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	79.27%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	87.07%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	82.20%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	82.44%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	72.93%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	88.29%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	86.10%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	88.29%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	88.29%
10	3	512	1.06×10^{-5}	0.580	33	48	Pruned	–
11	2	512	4.04×10^{-5}	0.582	5	48	Completed	85.61%
12	2	2048	5.26×10^{-6}	0.588	21	8	Pruned	–

continued on next page

Table A.3 (continued)

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
13	2	512	2.87×10^{-5}	0.727	9	48	Completed	87.80%
14	0	2048	1.14×10^{-6}	0.528	7	8	Pruned	–
15	2	2048	6.78×10^{-5}	0.512	27	8	Completed	87.56%
16	2	512	2.43×10^{-5}	0.700	25	48	Completed	85.85%
17	2	512	3.61×10^{-4}	0.502	21	48	Completed	87.80%
18	2	512	3.50×10^{-4}	0.505	5	128	Completed	88.29%
19	1	512	3.44×10^{-4}	0.597	11	48	Completed	90.24%

A.1.4 ResNet-50

Table A.4: All 20 clarifier Optuna trials for ResNet-50, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	82.44%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	87.07%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	78.54%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	81.22%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	70.98%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	83.66%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	85.37%
8	2	2048	9.08×10^{-6}	0.572	22	8	Pruned	–
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	78.05%
10	2	2048	6.20×10^{-3}	0.629	14	96	Pruned	–
11	2	2048	3.77×10^{-4}	0.875	17	96	Completed	89.02%
12	0	2048	3.46×10^{-4}	0.788	25	96	Completed	86.10%
13	2	2048	4.25×10^{-4}	0.925	17	96	Completed	87.32%
14	2	2048	1.21×10^{-3}	0.891	17	96	Pruned	–
15	2	2048	5.07×10^{-4}	0.745	11	24	Completed	88.54%
16	3	2048	2.50×10^{-3}	0.717	14	24	Pruned	–
17	2	512	6.73×10^{-4}	0.778	24	32	Completed	84.63%
18	2	2048	2.35×10^{-4}	0.728	11	16	Completed	87.80%
19	1	2048	5.91×10^{-4}	0.910	30	96	Completed	86.34%

A.1.5 ConvNeXt-Tiny

Table A.5: All 20 clarifier Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	42.93%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	50.00%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	42.93%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	89.76%
4	1	512	2.91×10^{-6}	0.931	27	96	Pruned	–
5	1	512	2.92×10^{-3}	0.800	30	16	Completed	42.93%
6	2	2048	1.38×10^{-3}	0.600	23	96	Completed	40.49%
7	2	1024	4.34×10^{-3}	0.629	28	48	Pruned	–
8	1	2048	1.22×10^{-5}	0.914	17	64	Completed	89.02%
9	1	512	2.91×10^{-6}	0.931	27	96	Pruned	–
10	2	512	5.13×10^{-5}	0.513	8	48	Completed	89.27%
11	3	2048	8.31×10^{-6}	0.618	33	64	Pruned	–
12	2	512	4.27×10^{-4}	0.647	11	48	Completed	88.78%
13	2	2048	4.94×10^{-6}	0.529	13	128	Pruned	–
14	0	512	2.24×10^{-6}	0.528	21	48	Pruned	–
15	2	512	8.07×10^{-5}	0.663	5	8	Completed	89.27%
16	3	512	4.95×10^{-5}	0.504	14	48	Completed	89.02%
17	1	2048	5.65×10^{-5}	0.560	21	128	Completed	90.73%
18	1	2048	1.07×10^{-4}	0.508	23	48	Completed	89.76%
19	1	512	2.11×10^{-5}	0.699	26	128	Pruned	–

A.1.6 Inception-v3

Table A.6: All 20 clarifier Optuna trials for Inception-v3, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	80.49%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	89.51%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	48.78%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	83.41%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	68.78%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	85.61%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	85.61%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	85.85%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	88.29%
10	2	2048	6.20×10^{-3}	0.629	14	96	Pruned	–
11	2	2048	2.55×10^{-4}	0.877	6	96	Completed	86.59%
12	3	512	2.74×10^{-4}	0.954	17	96	Completed	85.12%

continued on next page

Table A.6 (continued)

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
13	2	2048	4.06×10^{-4}	0.605	23	96	Completed	85.37%
14	3	512	7.80×10^{-4}	0.804	15	96	Pruned	–
15	3	512	1.36×10^{-5}	0.629	22	96	Pruned	–
16	2	2048	3.63×10^{-4}	0.877	33	96	Completed	85.12%
17	1	2048	1.69×10^{-3}	0.758	40	96	Pruned	–
18	2	2048	3.20×10^{-3}	0.512	6	128	Pruned	–
19	3	512	1.08×10^{-5}	0.890	8	96	Pruned	–

A.2 Aerobic Zones

A.2.1 ResNet-18

Table A.7: All 20 aerobic zone Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	73.02%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	75.07%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	46.92%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	77.71%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	71.26%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	78.89%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	77.71%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	78.01%
9	3	512	2.31×10^{-4}	0.838	5	96	Pruned	–
10	3	512	1.06×10^{-5}	0.580	33	48	Completed	80.35%
11	2	512	4.04×10^{-5}	0.582	6	48	Completed	75.95%
12	3	512	3.58×10^{-5}	0.719	37	48	Completed	82.11%
13	3	512	6.96×10^{-5}	0.684	26	48	Completed	83.28%
14	3	512	2.93×10^{-4}	0.706	24	48	Completed	83.87%
15	3	512	2.19×10^{-4}	0.512	26	48	Pruned	–
16	3	512	2.69×10^{-4}	0.664	28	8	Completed	74.19%
17	3	512	2.97×10^{-5}	0.643	19	16	Completed	78.59%
18	3	2048	1.09×10^{-3}	0.733	23	48	Pruned	–
19	1	512	1.54×10^{-5}	0.739	24	48	Completed	77.71%

A.2.2 EfficientNet-B0

Table A.8: All 20 aerobic zone Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	73.61%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	82.99%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	75.07%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	76.54%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	70.97%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	80.35%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	77.71%
8	2	2048	9.08×10^{-6}	0.572	22	8	Pruned	–
9	3	512	2.31×10^{-4}	0.838	5	96	Pruned	–
10	2	2048	6.20×10^{-3}	0.629	14	96	Completed	79.47%
11	2	512	8.03×10^{-4}	0.562	14	48	Completed	80.94%
12	2	2048	8.95×10^{-4}	0.716	38	96	Completed	82.11%
13	2	2048	4.06×10^{-4}	0.578	23	96	Completed	78.89%
14	2	2048	6.44×10^{-4}	0.746	33	96	Completed	81.52%
15	2	1024	4.22×10^{-5}	0.610	34	96	Pruned	–
16	2	1024	5.72×10^{-4}	0.731	17	96	Completed	80.65%
17	2	512	2.94×10^{-3}	0.567	34	96	Completed	74.78%
18	2	2048	8.30×10^{-3}	0.774	22	128	Pruned	–
19	0	2048	3.69×10^{-3}	0.827	35	96	Pruned	–

A.2.3 DenseNet-121

Table A.9: All 20 aerobic zone Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	71.55%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	79.47%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	74.19%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	77.13%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	74.19%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	78.30%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	78.89%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	82.11%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	78.89%
10	2	2048	1.45×10^{-5}	0.571	30	24	Completed	81.52%
11	2	2048	1.31×10^{-6}	0.671	26	8	Pruned	–
12	2	2048	4.57×10^{-5}	0.713	37	24	Completed	77.42%

continued on next page

Table A.9 (continued)

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
13	2	2048	8.32×10^{-6}	0.565	20	24	Pruned	–
14	2	2048	3.42×10^{-4}	0.532	29	24	Completed	77.71%
15	2	1024	2.38×10^{-6}	0.540	32	48	Pruned	–
16	0	2048	2.36×10^{-5}	0.637	28	32	Pruned	–
17	2	2048	2.00×10^{-6}	0.510	14	8	Pruned	–
18	2	512	3.99×10^{-5}	0.556	33	16	Completed	79.18%
19	2	2048	3.16×10^{-4}	0.742	20	8	Completed	84.46%

A.2.4 ResNet-50

Table A.10: All 20 aerobic zone Optuna trials for ResNet-50, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	58.36%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	80.35%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	46.92%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	69.79%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	71.85%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	74.19%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	75.07%
8	2	2048	9.08×10^{-6}	0.572	22	8	Pruned	–
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	76.54%
10	2	2048	6.20×10^{-3}	0.629	14	96	Pruned	–
11	2	2048	2.55×10^{-4}	0.877	6	96	Completed	78.30%
12	2	2048	8.95×10^{-4}	0.832	39	96	Completed	80.35%
13	2	2048	4.06×10^{-4}	0.604	23	96	Completed	81.82%
14	2	2048	3.59×10^{-4}	0.659	26	96	Completed	74.78%
15	0	2048	1.69×10^{-3}	0.541	20	8	Pruned	–
16	2	1024	3.17×10^{-4}	0.646	13	96	Completed	80.65%
17	2	2048	2.52×10^{-4}	0.610	10	96	Completed	79.18%
18	2	2048	1.15×10^{-3}	0.684	15	48	Completed	76.54%
19	2	2048	9.04×10^{-3}	0.594	35	96	Pruned	–

A.2.5 ConvNeXt-Tiny

Table A.11: All 20 aerobic zone Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	40.76%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	46.92%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	40.76%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	80.65%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	73.90%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	82.40%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	82.11%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	86.51%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	80.06%
10	2	2048	1.45×10^{-5}	0.571	30	24	Completed	81.23%
11	2	512	4.04×10^{-5}	0.582	5	48	Pruned	–
12	2	2048	5.26×10^{-6}	0.588	21	8	Completed	81.82%
13	2	512	1.00×10^{-5}	0.543	40	48	Pruned	–
14	2	512	4.75×10^{-6}	0.718	8	48	Pruned	–
15	2	2048	6.78×10^{-5}	0.512	27	8	Completed	87.39%
16	2	2048	7.11×10^{-6}	0.741	30	32	Pruned	–
17	2	2048	8.50×10^{-5}	0.503	27	16	Completed	87.39%
18	2	2048	6.09×10^{-5}	0.728	23	16	Completed	85.34%
19	2	1024	2.19×10^{-6}	0.534	33	8	Pruned	–

A.2.6 Inception-v3

Table A.12: All 20 aerobic zone Optuna trials for Inception-v3, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	73.61%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	75.66%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	73.02%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	78.01%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	73.31%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	77.71%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	80.94%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	78.59%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	76.54%
10	1	512	1.34×10^{-5}	0.797	11	32	Completed	74.19%
11	2	512	6.30×10^{-6}	0.541	28	48	Pruned	–
12	2	2048	5.26×10^{-6}	0.662	21	8	Completed	75.07%

continued on next page

Table A.12 (continued)

Trial	H	N	**LR**	β_1	**Step**	**Batch**	**Outcome**	**Dev. acc.**
13	2	512	2.85×10^{-4}	0.588	33	8	Completed	82.40%
14	3	512	3.22×10^{-5}	0.716	23	32	Completed	79.47%
15	2	512	2.25×10^{-4}	0.682	37	8	Completed	79.47%
16	0	512	1.79×10^{-4}	0.568	23	8	Completed	81.52%
17	0	512	5.30×10^{-4}	0.526	16	8	Pruned	–
18	0	512	2.10×10^{-4}	0.551	25	64	Completed	80.65%
19	0	512	5.33×10^{-5}	0.624	18	8	Completed	80.35%